Applying DeepLabV3+ Semantic Segmentation Model to Enhance Reinforcement Learning Training Time in 2D Platformer Games

---

A Thesis Presented to
The Faculty of the College of Computer Studies
Tarlac State University
Tarlac City

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Science in Computer Science

---

by:

Kian Enrico G. Martinez
Karl Christian C. Panigbatan
Christian Neo Y. Calma

August 2024

TARLAC STATE UNIVERSITY
COLLEGE OF COMPUTER STUDIES

TCP FORM 14

APPROVAL SHEET

This Thesis of **Kian, Karl,** and **Christian** entitled **"Applying DeepLabv3+ Semantic segmentation Model to Enhance Reinforcement Learning Training Time in 2D Platformer Games**", which is prepared and submitted in partial fulfillment of the requirements of the degree **Bachelor of Science in Computer Science** is hereby approved and accepted.

MR. LARSEN DIGNOS
Technical Adviser

PANEL COMMITTEE

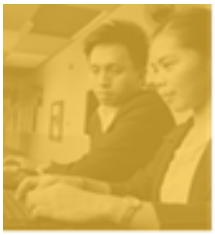
MS. REGINA ARCEO
Chairman

MR. KWINNO PINEDA
Member

MR. NOMAR LAPITAN
Member

Accepted and approved in partial fulfillment of the requirements for the degree **Computer Science**.

DR. ALVINCENT DANGANAN
Dean, College of Computer Studies
August 2025



TARLAC STATE UNIVERSITY
OFFICE OF UNIVERSITY RESEARCH DEVELOPMENT

ORD-TC-25-082

CERTIFICATE
OF COMPLIANCE

This is to certify that the research details below have been evaluated by online anti-plagiarism software. The contents and material were found satisfactory and within the permissible limit of content copied.

Name of Author/s	
Kian Enrico G. Martinez	Christian Neo Y. Calma
Karl Christian C. Panigbatan	

Applying DeepLabV3+ Semantic Segmentation Model to Enhance Reinforcement Learning Training Time in 2D Platformer Games

Submission ID	College/Agency
26063108860284	COLLEGE OF COMPUTER STUDIES

Results:

Submission	Similarity Index	AI Index
First	10%	—
Second	—	—
Third	—	—

Certified by:

DR. ROBERT V. MARCOS
Director, OURD

Date: 08/20/2025

ABSTRACT

This paper addresses the challenge of reducing training time in reinforcement learning for 2D platformer games by integrating DeepLabV3+ semantic segmentation into the training pipeline. This study implemented DeepLabV3+ to preprocess game visuals from Super Mario Bros., simplifying the complex environments into easily interpretable segments for the reinforcement learning agent. Through extensive testing, the researchers found that the segmentation model effectively isolated key game elements, which enabled the agent to focus on relevant features and thereby accelerate its training process. As a result, the enhanced training framework not only reduced computational requirements but also improved the overall performance and efficiency of the learning agent. This successful application of DeepLabV3+ highlights its potential to optimize reinforcement learning in video games, offering a promising avenue for faster, more efficient AI development in complex interactive environments.

DEDICATION

First and foremost, we dedicate this work to God, whose guidance, strength, and blessings made this journey possible. His grace has been our source of perseverance through every challenge and accomplishment.

To our parents, thank you for your unwavering love, encouragement, and sacrifices. Your constant support has been the foundation of our growth, both academically and personally.

To our friends, who stood by us during long nights and stressful deadlines—your encouragement, humor, and shared struggles made the experience lighter and more meaningful.

And finally, to everyone who supported us throughout this project—our mentors, classmates, and those who believed in our capabilities—your contributions, no matter how big or small, have played an important role in bringing this work to completion. This achievement is as much yours as it is ours

ACKNOWLEDGMENT

We extend our sincere appreciation to our Technical Adviser, **Sir Larsen Dignos**, whose technical expertise and consistent guidance played a key role in the development and execution of this study. Your input on the system architecture and workflow was instrumental in refining our approach and strengthening the core of our project.

We also extend our heartfelt thanks to our Thesis Subject Adviser, **Sir Julius Caesar Ramos**, for your academic guidance and patience in addressing our questions throughout the process. Your steady support and constructive feedback played a vital role in shaping the direction, clarity, and overall quality of our research work.

We are also thankful to our Thesis Panel Member, Chairperson **Ma'am Regina P. Arceo**, **Sir Kwinno L. Pineda**, and **Sir Nomar R. Lapitan** for offering valuable critiques and objective feedback which contributed to the refinement of this thesis project..

Above all, we are deeply grateful to God. Through every obstacle and moment of uncertainty, His presence provided us with the strength, patience, and determination to complete this journey. This accomplishment would not have been possible without His guidance and grace.

TABLE OF CONTENTS

LISTS OF FIGURES

LISTS OF TABLES

1 INTRODUCTION

A. Project Context

Artificial intelligence has proven to be useful over the years and reinforcement learning has proven to have great potential in training agents to fix and solve complex problems by interacting with different environments [1]. Also, in 2D platformer games, reinforcement learning makes the agents learn the game mechanics by having it navigate through it and complete its programmed objectives without predefined restrictions [2]. Meanwhile, a significant challenge in its longer training time cycle has made it problematic for the agent to learn more efficiently [3]. In computer vision, a semantic segmentation model called DeepLabV3+ has proven to be successful in isolating and segmenting key elements that is within the images provided [4]. By adding the DeepLabV3+ semantic segmentation model into the reinforcement learning pipeline for training in 2D platformer games, it may reduce the training time required for it to learn efficiently, allowing it to understand and successfully interact with key elements within the game thus resulting in enhancing its training time for the reinforcement learning agent [5].

The normal approach for reinforcement learning in the 2D platformer game environment to make decisions is for it to learn from pixelated data, thus making it longer for the training cycle to reach its peak performance and including its high computational demands [1]. The reinforcement learning agent's long training cycle may determine its learning process interpretation of the whole game scene, it includes objects, platforms, obstacles, and rewards [2]. These complex visuals in the 2D game proves to be a challenge for the agent's learning and resulting it to have a much slower rate in its decision-making

moves [3]. Meanwhile, the issue of extensive time required for the agent to learn efficiently in each areas of the 2D game scene proves to be as important as its learning process [5]. This factor on its training time gives an increase to its training time, computational costs, and finally the development cycle for the agent [3]. The given issue on this gives a priority for its approach in key elements on the game, elements such as platforms, enemies, and obstacles which becomes a challenge in the agent's ability to maximize its learning on the environment [4]. Meanwhile, the application of a semantic segmentation model to further simplify the visual input may help reduce the agent's training time [5].

A research study conducted in Madrid titled "Exploiting semantic segmentation to boost reinforcement learning in video game environments" helped in the implementation of the DeepLabV3 model with its backbone ResNet-50 to help it improve its learning capabilities in 2D platformer games by allowing the segmentation model to simplify key game elements to allow the agent to learn faster across multiple game levels [5]. As the study continued, the game Super Mario Bros was tested and it helped the agent learn better and achieve positive results in fewer training episodes across different game levels compared to the traditional way of training the agent in this field [5]. Even though the approach required a segmentation dataset, this allowed the agent to learn faster without changing the agent's algorithm.

In general, DeepLabv3 is a popular segmentation model in terms of performance, but it's missing the decoder module that can be found in DeepLabv3+[10]. This feature only limits the ability of DeepLabv3 to accurately define the object edges in a more complex scene. With this, DeepLabv3+ works better for reinforcement learning tasks that will require accurate object identification.

By choosing to integrate the DeepLabv3+ into the agent's pipeline, the environment can be segmented into a simpler approach [10]. This can allow the agent to only focus on key elements in the game (platform, enemies, obstacles) , accelerating its learning process and reducing the training time [5].

This study involved the use of a semantic segmentation model and reinforcement learning agent. The researchers used DeepLabv3+ for the segmentation model. Compared to DeepLabv3, the DeepLabv3+ offers enhanced functionalities for a more accurate detection, which also introduces the decoder module that the DeepLabv3 lacks [10]. OpenAI Gym will also be utilized to set up the 2D platformer game environment. Since it is a package that supports emulation for different video game consoles, and provides functionalities to interact with games, like memory editing or input simulation for the game [11]. As for the reinforcement learning algorithm, the researchers used the Deep Q-Learning (DQN) algorithm. Since this algorithm is much more robust and it is easier to implement [5].

The normal process for developing platformer games is a formal pipeline: pre-production (concept design and documentation), production (coding, level design, AI behavior, and asset creation), testing (bug fix and optimization), and post-production (release and updates) [29]. Value in our system, which incorporates semantic segmentation with reinforcement learning, would reside in the latter stages of testing and production. It would enable the creation of more intelligent, world-aware agents and non-player characters (NPCs) through the abstraction of raw visual data to semantically meaningful input, simplifying the reinforcement learning process. It can reduce manual scripting development time in behavior and enhance the ability of the agent to learn from level to

level. It can also contribute to automatic testing by indicating visual or functional inconsistencies using stringent segmentations over the overlay, raising the degree of efficiency and consistency of game design and logic in the process. These challenges and integration opportunities are consistent with those highlighted in the study by K. Souchleris, et al, *"Reinforcement Learning in Game Industry—Review, Prospects and Challenges"* [29], which emphasizes the practical difficulties in adopting reinforcement learning for real-time environments, model generalization, and the development of adaptable, efficient NPC behavior.

Hence, this thesis proposed integrating DeepLabV3+ Semantic segmentation with reinforcement learning to enhance training time for agents in 2D platformer games. In addition, this study won't involve a comparative research method; instead the researchers used the findings of published research to compare the semantic models. The new decoder structure of the DeepLabv3+ highlights its capabilities in comparison to the DeepLabv3 model which lacks the decoder module making the DeepLabv3+ better suited for RL environments where precise object recognition is important for the decision making of the agent. By breaking down game environments into segmented, interpretable parts, the RL agent can focus on learning critical elements more efficiently, thus reducing training time. Also, the improved interpretability of this model will help enhance the agent's ability to interpret game states accurately. The researchers also concluded that the use of DeepLabv3+ as the segmentation model is mainly because of its enhanced performance in segmenting tasks [10]. The method of using the DeepLabV3+ segmentation model highlights the missing functionalities of the other segmentation model methods to be used

in complex game environments, this also offers a more efficient way for a faster reinforcement learning agent training.

B. Purpose and Description

This study aims the use of the DeepLabV3+ segmentation model to be integrated on the reinforcement learning agent's pipeline to help it achieve better results in its training time on 2D platformer games. The use of this method in can help simplify the input of key game elements to the agent, elements such as obstacles, enemies, and platforms. Helping the agent feed on simplified data input can enhance the agent's decision making and be able to remove unnecessary movements in other areas of the game and by removing this movements, it can prioritize areas that can produce better results in its learning process and reduce its training time.

As for the beneficiaries of this study, it will be both game developers since this study involves the use of a 2D platformer game and also AI researchers where a reinforcement learning agent is also used. This study will focus on offering methods to advance AI implementation without heavy resources to rely on to for complex game environments. This will also be able to give game developers options to create a better and more intelligent game reinforcement learning agents.

C. Objectives

The general objective of this study is to prove that Semantic segmentation can enhance reinforcement learning in 2D video games to be able to learn efficiently and

provide a more positive outcome in the training time needed for the agent to fully navigate through the game.

Specifically, the study aims to achieve the following:

1. To compare DeepLabv3+ with other semantic segmentation algorithms, specifically DeepLabv3, by analyzing findings from published research.

2. To develop and train a semantic segmentation model for DeepLabV3+ to make smarter gameplay decisions using segmented output from DeepLabV3+

3. To evaluate the improvement in training time, a quantitative assessment will be conducted using Mean Average Precision (mAP) and Mean Intersection over Union (mIoU) to measure the accuracy of the semantic segmentation model. Additionally, a qualitative assessment will be carried out through a Likert scale questionnaire to validate the findings with IT experts in terms of:

   3.1. Quality of Segmentation

   3.2. Ability of RL Agent to Clear Levels

   3.3. Impact on Training Efficiency and Overall Performance

D. Scope and Limitation

This research study is based on the application of a segmentation model on 2D Platformer games with our chosen reinforcement learning agent on the game Super Mario Bros. A previous study titled “Exploiting semantic segmentation to boost reinforcement learning in video game environments” proved that by using a segmentation model, the training time of the agent has been reduced by using DeepLabV3 segmentation model. With this, the researchers' approach used an enhanced version of the DeepLabV3, which is the DeepLabV3+. Our research study proved that our approach is better than the previous

study in terms of reducing the training time of the reinforcement learning agent with the use of DeepLabV3+ on the same game, Super Mario Bros. By improving the environment understanding, the DeepLabV3+ can segment different game elements such as the **platform, enemies, and obstacles from the background** which provided the agent a clearer representation of the environment that allowed the agent to make more informed decisions. A dataset generator is also used in this research study to generate the frames of the game environment. Also, by effectively segmenting the key game components, the model reduced the complexity of the state space which led to a faster convergence times during training. Our method of using the DeepLabV3+ captures multi-scale features effectively providing us with an enhanced feature extraction and enables the agent to recognize and react to different objects sizes and types, improving gameplay strategies.

Moreover, the study is only focused on 2D platformer games and not with other 2D game types such as 2D Board games like chess, monopoly, and scrabble. Time constraints may also limit the depth of analysis and refinement of both the learning agent and the segmentation model because a longer duration for the both could give a more comprehensive result. These limitations must be taken into consideration when interpreting the results of the study.

2 REVIEW OF RELATED LITERATURE AND STUDIES

A. Discussion of Models

In the context of this study, the first example of a Semantic segmentation model is the Fully Convolutional Networks (FCNs). FCN model is the first choice for most semantic segmentation systems. However, the final predictions in the FCN model are low-resolution predictions. In order to solve this problem, a skip structure is introduced to refine the final predictions to generate a more detailed feature score map. In this way, FCN can produce more accurate and detailed semantic segmentations [13].

Following the previous model, U-Net is a convolutional neural network that was developed for Semantic segmentation and it can be divided in two parts, The first is the contracting path that uses a typical CNN architecture and the second part called the expansive path, in which each stage samples the feature map using 2 x 2 up-convolution. At the final stage of U-Net, an additional 1 x 1 convolution is applied to reduce the feature map to the required number of channels and produce the segmented image. This results in a network resembling a u-shape and, more importantly, propagates contextual information along the network, which allows it to segment objects in an area using context from a larger overlapping area [14]

The next model is the Segnet, it has an encoder network and a corresponding decoder network, followed by a final pixel wise classification layer. With the use of this method, it can initialize the training process from weights trained for classification on large datasets. For Segnet, each encoder layer has a corresponding decoder layer. The final decoder output is fed to a multiclass soft-max classifier to produce class probabilities for each pixel independently [15].

In addition to this, the Mask R-CNN segmentation method extends Faster R-CNN by adding a branch for predicting an object mask in parallel with the existing branch for bounding box recognition. Mask R-CNN is simple to train and adds only a small overhead to Faster R-CNN [16].

However, the PSPnet segmentation model offers a multiscale network. that applies a pyramid pooling module to the field of semantic segmentation, so that it can better learn the global context information of the scene and effectively improve the segmentation accuracy [18].

Meanwhile, the Attention U-Net offers a different way in segmentation, by making use of the attention gate. An attention gate is a unit that trims features that are not relevant to the ongoing task. Each layer in the expansive path has an attention gate through which the corresponding features from the contracting path must pass through before the features are concatenated with the upsampled features in the expansive path. Repeated uses of the attention gate after each layer significantly improves segmentation performance without adding excessive computational complexity to the model [14].

Following this, the DeepLabV3 semantic segmentation model relies on atrous convolutions to control the receptive field of the network without requiring additional parameters [5]. This model also includes what the authors called an Atrous Spatial Pyramid Pooling (ASPP) module to extract the information from different scale atrous convolutions. Also, the test of comparing these three backbones-Resnet-50, ResNet-101, and MobileNetV3 for the DeepLabV3, the ResNet-50 is used due to its speed and accuracy over the other backbones for the segmentation mode with learning rate of 0.001 and decay adjustments at specific epochs. The model in the study [5] also used pre-trained weights

from the COCO2017 dataset, improving accuracy by about 2.5% over training from scratch. The evaluation also stated that the mean Intersection over Union (mIoU) was used to assess model accuracy, with ResNet-50 achieving mIoU on the synthetic dataset. However, considerations may be needed when using the DeepLabV3 segmentation model, since it has a class dependency in learning, relying on isolated classes for semantic understanding will lead to poor reinforcement learning outcome and it may perform inconsistently across visually varied game levels [4].

Another model for Semantic segmentation is the DeepLabV3+ which is by far one of the best semantic segmentation algorithms currently. In the encoding phase, DeepLabV3+ uses the optimized and improved Xception network as the backbone network to extract features, and the optimized and improved Xception network structure to further enhance the performance with faster computation for real-world applications and complex environments [4]. DeepLabV3+ also uses an encoder-decoder architecture to capture finer details of the environment, this decoder module can can also capture spatial details more accurately which can improve the reinforcement learning outcomes compared to DeeplabV3's use of isolated classes for semantic understanding [4]. The encoder-decoder architecture also allows the model to capture detailed contextual information giving it a more flexible option for transfer learning [4]. The components of the new encoder-decoder architecture of DeepLabv3+ uses the deepLabv3 model as the encoder and the output from the last feather map (before logits) is used as the input to the encoder-decoder structure. Another feature map from the shallow layers (closer to the input) of the backbone is taken and subjected to (1×1) convolution to reduce the number of channels. This low-level feature map is rich in spatial information. Also, the output from the encoder is first

bilinearly upsampled by a factor of 4. Both feature maps from (1) and (2) are concatenated and passed through two (3×3) convolution blocks. The merging allows the refinement of the encoder feature map with high spatial information (object boundary). Finally, to generate predictions, the refined feature map is bilinear upsampling by a factor of 4 [10]. In comparison to the DeepLabv3, These details are related to the architectural changes made to DeepLabv3+. This strength is what highlights the DeepLabV3+ to its predecessor, the DeepLabV3 making it a better choice for segmentation tasks.

Table 1 Functional and Feature Matrix

	1	2	3	4	5	6	7
Fully Convolutional Networks (FCNs)	✓	X	X	X	X	X	X
U-Net	✓	✓	X	✓	X	X	X
Segnet	✓	✓	X	✓	X	X	X
Mask R-CNN	✓	✓	X	✓	X	X	✓
PSPNet	✓	✓	X	✓	✓	X	X
Attention U-Net	✓	✓	X	✓	X	✓	X
DeepLabv3	✓	X	✓	✓	✓	X	X
DeepLabv3+	✓	✓	✓	✓	✓	✓	✓

1. Skip Connections - Preserves spatial details which are crucial for accurately segmenting game elements like character and backgrounds.
2. Encoder-Decoder Architecture - Effectively captures game features at different scales, enabling better reconstructions and segmentation of 2D game scenes.
3. Atrous Convolutions - helps in gathering contextual information from surrounding pixels, which is important for segmenting objects in various sizes and complexities found in 2D games.
4. CRFs for Refinement - Enhances the segmentation results by ensuring smooth transitions and coherent boundaries between game elements, like character outlines and environmental features.
5. Pyramid Pooling - gives the model the function to analyze game frames, thus enhancing the segmentation of 2D game objects.
6. Attention Mechanisms - mainly target important parts within the game frame that will affect the model's segmentation properly.
7. Multi-Scale Predictions - make improvements on the model's overall performance by playing it in different resolutions.

Table 1 features the comparison of different algorithms in terms of segmenting 2D platformer game elements. The feature skip connection contributes to preserving important details, making the model recognize specific parts of the game easily, while the encoder-decoder feature allows for a more simplified reconstruction of the game environment. Atrous convolutions help in analyzing pixels, which then improves the model's ability to detect different objects. CRFs are used for the refinement of the quality of segmentation and to smooth the boundaries between game elements. Pyramid Pooling lets the model

view the scene from multiple zoom levels, while Attention Mechanism direct focus to key regions in the game, enhancing the segmentation of critical elements, Lastly Multi-Scale Predictions boost model performance by capturing details at different resolutions this helps accurately segment the diverse parts of the game scenes, which overall makes the models more effective at improving visual and interactive quality of 2D platformer games

DeepLabV3+ was the chosen algorithm because of its ability to capture both fine details and broader context, making it ideal for segmenting complex 2D platformer game environments. Its encoder decoder architecture can capture essential features at various scales while retaining precise object boundaries; this is what led the researchers in using the DeepLabV3+ since this architecture is not included in the DeepLabV3. Addition to that, its strong boundary refinement ensures accurate segmentation, enhancing the reinforcement learning agent's ability to interpret the environment and potentially improve learning speed and decision-making performance. Our method of choosing DeepLabV3+ paired with ResNet-50 as its backbone since it is more compatible with segmentation tasks regarding 2D Platformer games including the harder levels of 2D Platformer games as its speed can lead to faster learning cycles. Even though the Xception backbone provides an improved functionalities as the backbone of DeepLabV3+, it is more compatible for handling games with a required higher resolution.

The use of the DeepLabv3+ model helped us enhance the training time of the reinforcement learning agent or help it reduce the time needed for it to train in a more efficient way. Also, with our DeepLabv3+ model, we further simplified the Super Mario Bros game and labeled each element of the game for the agent to easily recognize it resulting in a faster training time compared to the other literatures mentioned in this study,

since their methods solely rely on the implementation of a learning agent in 2D Games specifically focused on 2D Platformer games without any additional method to help reduce the training time of the agent. However, there is a study that implemented a semantic segmentation model in Super Mario Bros. They used the DeepLabv3 model to reduce the training time of the agent and it gained positive results in terms of reducing the training time. Our approach with the DeepLabv3+ model helped us prove that we can further reduce that training time of the agent that is focused on 2D Platformer games, specifically the game Super Mario Bros.

Table 2 Literature Matrix

Reference	Description	Strength	Weakness
J. Montalvo, Á. García-Martín, and J. Bescós, "Exploiting semantic segmentation to boost reinforcement learning in video game environments," *Multimedia Tools and Applications*, vol. 82, no. 7, pp. 10961–10979, Sep. 2022, doi: 10.1007/s11042-022-13695-1.	This study focuses on enhancing the performance of reinforcement learning algorithms using the DeepLabv3 semantic segmentation model.	• Simplified input representation • Faster training of the RL • Higher and more efficient performance • Use of ResNet-50 as the backbone for the model to better segment 2D Platformer games	• Image input dependency • Dataset requirement for semantic segmentation • Computation time for the training time • Struggles with domain adaptation
Gomes, G., Vidal, C. A., Cavalcante-Neto, J. B., & Nogueira, Y. L. (2022). A modeling environment for reinforcement learning in games. *Entertainment Computing*, *43*, 100516. https://doi.org/10.1016/j.entcom.2022.100516	This study focuses on the use of Artificial Intelligence for Unity (AI4U) modeling environment to make prototypes of autonomous non-Player Characters (NPCs) for video games.	• Simplicity in facilitating quick prototyping with Unity • Able to support various implementations of AI algorithms • Successfully used in projects involving autonomous virtual	• Limited to Unity • Users may need time to adapt to new functionalities and features. • Some implementations may require significant computational resources. • Training the agent in more

		characters with emotions. ● Provides a structured framework for developing NPCs based on task environments.	complex games unlike 2D Platformer games impacts the training time of the agent.
Souchleris, K., Sidiropoulos, G. K., & Papakostas, G. A. (2023). Reinforcement Learning in Game Industry—Review, Prospects and Challenges. *Applied Sciences*, *13*(4), 2443. https://doi.org/10.3390/app13042443	This study focuses on the recent advances in the field of reinforcement learning (RL) as well as the present state–of–the–art applications in games.	• Game-based environments facilitate the creation of comprehensive and interactive settings for testing algorithms. • Growing interest in using RL and deep RL in gaming contexts, indicating active exploration and development in the field. • Successful algorithms in gaming contexts have the potential to solve real-world challenges, such as self-driving cars. ● Recent algorithms show promise in managing complex multi-centric decision-making tasks, enhancing their usefulness.	• Research has predominantly focused on either single complex environments (like AlphaGo) or simpler systems (like ATARI games), raising concerns about generalizability. • Current research may prioritize achieving high performance in limited settings rather than advancing general intelligence across diverse environments. • Simulated environments cannot fully replicate the complexities of real-world scenarios ● The comparison in training the agent in MOBA games between ATARI games proved to be a lot longer.
Kang, S., & Choi, J. (2022). Unsupervised	This study game UI segmentation	• Use of unsupervised	• Limited generalization

semantic segmentation method of user interface component of games. *Intelligent Automation & Soft Computing*, *31*(2), 1089–1105. https://doi.org/10.32604/iasc.2022.019979	technique that leverages unsupervised learning to enhance the analysis of game screens through vision-based machine learning algorithms	learning method • Training without extensive manual annotation • Reduces the need for additional data augmentation techniques, • Facilitates the extraction of meaningful semantic information from game images ● Application of UI Prototyping	• Dependence on Game Engine that relies heavily on the specific game engine ● The difference of using a GPU with high specifications between a lower specification GPU impacts the speed of the training time of the AI.
S. Karakovskiy and J. Togelius, "The Mario AI benchmark and competitions," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 55–67, Feb. 2012, doi: 10.1109/tciaig.2012.2188528.	The Mario AI Benchmark evaluates reinforcement learning algorithms and game AI in a platformer game setting. It is based on Infinite Mario Bros, a clone of Nintendo's Super Mario Bros, and serves as a testbed for AI capabilities in 2D platform games. Competitions have been held to test AI performance.	• Offers a customizable and diverse platform for testing AI algorithms. • Incorporates a tunable level generator to vary difficulty and diversity of challenges. • Supports learning-based and heuristic AI methods, enabling diverse participation. ● High-speed simulation makes it computationally efficient for research.	• Limited by computational approaches that might outperform others without complex learning (e.g., A* vs. learning algorithms). • Training time for agents can be substantial, especially for learning-based methods (e.g., 10,000 iterations for training in learning tracks). ● Challenges like dead ends were added later, exposing potential oversights in earlier benchmarks.
V. Sriram, "Automated playtesting of platformer games using	The study develops an Automated Playtesting (APT) tool using deep	• Cost-effective alternative to	• Agents do not emulate human playstyles,

reinforcement learning," 2019. doi: 10.17760/d20335186..	reinforcement learning (DRL) and curriculum learning to test 2D platformer games for quality assurance and game balance. The trained AI agent can identify gameplay challenges and suggest improvements.	human playtesting. • Provides quantitative and qualitative data through heatmaps and interaction logs. • Utilizes curriculum learning to gradually train agents on increasing difficulty. • Demonstrates significant time savings (434 playthroughs in 15 minutes by a trained agent).	focusing instead on maximizing rewards, which can diverge from typical human behavior. • Limited by the mechanics and environment considered during training; adding new features requires retraining. • Overfitting observed if training exceeds 150,000 steps.
B. Setiaji, E. Pujastuti, M. F. Filza, A. Masruro, and Y. A. Pradana, "Implementation of reinforcement learning in 2D based games using open AI gym," *2022 International Conference on Informatics, Multimedia, Cyber and Information System (ICIMCIS)*, Nov. 2022, doi: 10.1109/icimcis56303.2022.10017810	The paper explores the implementation of reinforcement learning (RL) in 2D platformer games using OpenAI Gym, aiming to improve game AI and automate gameplay testing.	• Demonstrates RL in an accessible 2D game environment. • Uses OpenAI Gym, a popular tool for RL. • Highlights adaptability and potential for automating game AI.	• Focused only on 2D games, limiting broader applications. • Requires significant computational resources for training. • Training time can be lengthy for complex environments.
C. Hu *et al.*, "Reinforcement learning with Dual-Observation for general video game playing," *IEEE Transactions on Games*, vol. 15, no. 2, pp. 202–216, Apr. 2022, doi: 10.1109/tg.2022.3164242.	The paper introduces a dual-observation reinforcement learning method, which enhances decision-making by using both game state and environmental context. This approach is effective in 2D games and improves generalization across various types of games.	• Improves generalization across 2D and other games. • Dual-observation enhances decision-making. • Outperforms traditional RL models.	• High computational resources needed due to dual-observation. • Longer training times due to the complexity of the model.

			• Performance can vary based on the game's complexity.

B. Conceptual Framework

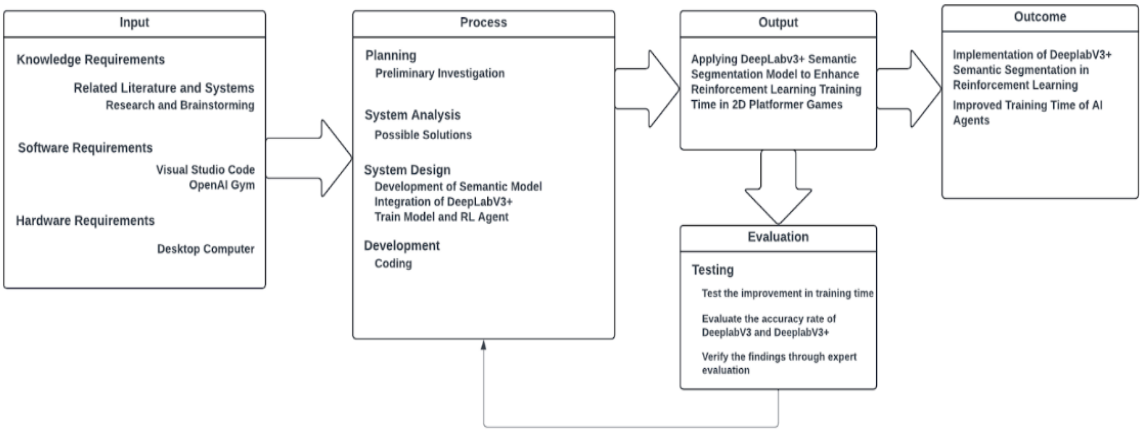


Figure 1 Conceptual Framework

As illustrated in Figure 1, the study began with the researchers familiarizing themselves with the required knowledge for this study. This includes topics such as Semantic segmentation, reinforcement learning, and also how to use the required software, in this case, OpenAI Gym, Visual Studio Code, Cursor AI, and SMB Utility. These knowledge requirements are essential to understand how to apply DeepLabV3+ semantic segmentation with reinforcement learning algorithms in 2D platformer games. The research and brainstorming phase involved exploring existing literature and systems to identify potential approaches and challenges in implementing this integration to find potential ways and obstacles in establishing this integration.

The development process started with preliminary research, followed by coming up with alternative solutions, designing the game environment, and coding. On the duration

of the preliminary examination, The researchers thoroughly researched ways on how to integrate the DeepLabV3+ semantic segmentation model with reinforcement learning. Onwards to system analysis where each process are important in analyzing potential solutions and creating an effective strategy on how to proceed with the study's objectives. The system design phase is where the researchers focused on drafting layouts, getting the sprites, the creation of the game environment itself, and finally the development of the framework needed to be used for the integration of the DeepLabV3+. Afterwards, the development phase included extensive coding of the system that includes the study's aim to use both semantic segmentation pipeline and reinforcement learning agent.

Each of these phases are important to reach the target output of applying the DeepLabV3+ semantic segmentation model to help enhance the reinforcement learning agent's training time. As for the chosen evaluation of the agent's performance across different levels, the comparison will be made to measure the training efficiency with or without the semantic segmentation. This comparison testing method will help validate the system's effectiveness in the enhanced training efficiency of the agent. The study's ultimate goal was to show that semantic segmentation could be successfully implemented in reinforcement learning and that AI agents could train faster in 2D platformer games.

C. Definition of Terms

Table 3 Definition of terms

Reinforcement learning	Reinforcement learning is an interdisciplinary area of machine learning and optimal control concerned with how an intelligent agent should take actions in a dynamic environment in order to maximize a reward signal. An RL agent will be used for the testing stage of this study to prove if our approach on reducing its training time is effective.

2D Games	2D games are digital entertainment products based on two-dimensional flat art that only has width and height measurements. This is the environment that we are going to use to train the RL agent.
Enemies	Enemies are characters or objects that challenge the players by preventing players to reach the goal , requiring players to either dodge or defeat them to move forward and reach the goal
Obstacles	Obstacles are stationary or moving objects that hinder the player's progress, requiring the players to jump over, avoid or find another path to reach continue through the level
Semantic segmentation	Semantic segmentation is a computer vision technique that involves dividing an image into distinct regions or segments, each representing a specific object or area of interest.
Platforms	Platforms are objects in the game that players can interact with, they can be used to stand on or are designed as obstacles
Sprite	Sprites are images that represent the visual aspect of the game, this includes the characters, objects, and VFX.
NPC (Non-Player Character)	An NPC is any character in a game that is not under the player's control is known as an NPC. NPCs in 2D platformers might be neutral people, allies, or foes who could influence or interact with the player's path
AI (Artificial Intelligence)	The programming that controls how NPCs and other game elements behave is referred to as artificial intelligence (AI). AI frequently determines how adversaries move, attack, and react to the player's actions in 2D platformers, which adds difficulty and interest.
Weighted Mean	Weighted Mean is an average computed by giving different weights to some of the individual values
MAP (Mean Average Precision)	MAP is a metric used to measure the performance of a model for tasks such as object detection tasks and information retrieval
Mean Intersection over Union (IoU)	mIoU Is the area of overlap between the predicted segmentation and the ground truth divided by the area of union between the predicted segmentation and the ground truth

Likert Scale	A Likert scale is a psychometric rating scale commonly used in surveys to measure opinions, attitudes, or behaviors

3 TECHNICAL BACKGROUND

A. Software Development Requirements

Table 4 Software Requirements

SOFTWARE	DESCRIPTION
OpenAI Gym	OpenAI Gym is a toolkit designed for developing machine learning related programs.
Jupyter Notebook	Project Jupyter is a project to develop open-source software, open standards, and services for interactive computing across multiple programming languages.
PyTorch	PyTorch is a versatile and powerful library that enables researchers and developers to build and train models efficiently.
Python	Python is a programming language that is widely used in web applications, software development, data science, and machine learning (ML).
SMB Utility	SMB Utility is a powerful and user-friendly ROM hacking tool designed to edit levels in the original NES Super Mario Bros. game.

OpenAI Gym was utilized to create and import the 2D Platformer games. Jupyter Notebook was utilized as the Integrated Development Environment (IDE). Pytorch was used to implement the semantic segmentation models. Python was used to be the programming language and its scripts have been written to. Since it offered numerous advantages for research and data analysis, particularly in fields like machine learning, data science, and reinforcement learning. Lastly, the SMB utility software was used to help us customize the Super Mario Bros game.

B. Hardware Development Requirements

Table 5 Hardware Requirement

HARDWARE	SPECIFICATION

Laptop	Processor: 12th Gen Intel(R) Core(TM) i5-12500H 3.10 GHz Installed RAM : 16.0 GB GPU: RTX 3060 Windows 11 64-bit

The laptop was used to develop the entirety of the project including the development of the game, application of the algorithms and the testing of the algorithms. We utilized a laptop since it offers a compactness and portability for us to bring anywhere for us to proceed with our study even if we are outside of our houses. Also, one of the reasons why we utilized a laptop was its high specifications since training the reinforcement learning agent requires a higher specification device. For this, we chose Intel Core i5 as our processor paired with the Windows 11 operating system. For more performance, the RTX 3060 GPU was more than enough to handle the workload of our study paired with 16 GB ram for more positive performance in training our reinforcement learning agent.

D. Sources of Data for Development and Testing

Table 6 Sources of Data

Sources	Description
OpenAI Gym	A toolkit for reinforcement learning to be utilized by the researchers in creating the game environment of Super Mario Bros.
Dataset Generator	This generator is programmed to generate synthetic frames of the Super Mario Game by combining the sprites provided to make the images look like the frames taken from the videogame.
Mendeley	The researchers use this to gather more documentation for planning and references. Mendeley serves as a search engine for gathering similar

	literature related to this research study.
Google Scholars	The researchers use this as means to gather more documentation for planning and references. Google scholar is a search engine that indexes scholarly literature across a variety of disciplines
Sprites	Sprites will be used to generate synthetic frames of the original games that the researchers will use in the study.
IEEE Website	Institute of Electrical and Electronics Engineers (IEEE), the researchers used this to gather documentations and references
Library Genesis	LibGen is a shadow library, which means it provides free access to content that is usually paywalled or not available elsewhere.

4 DESIGN AND METHODOLOGY

A. Research Design

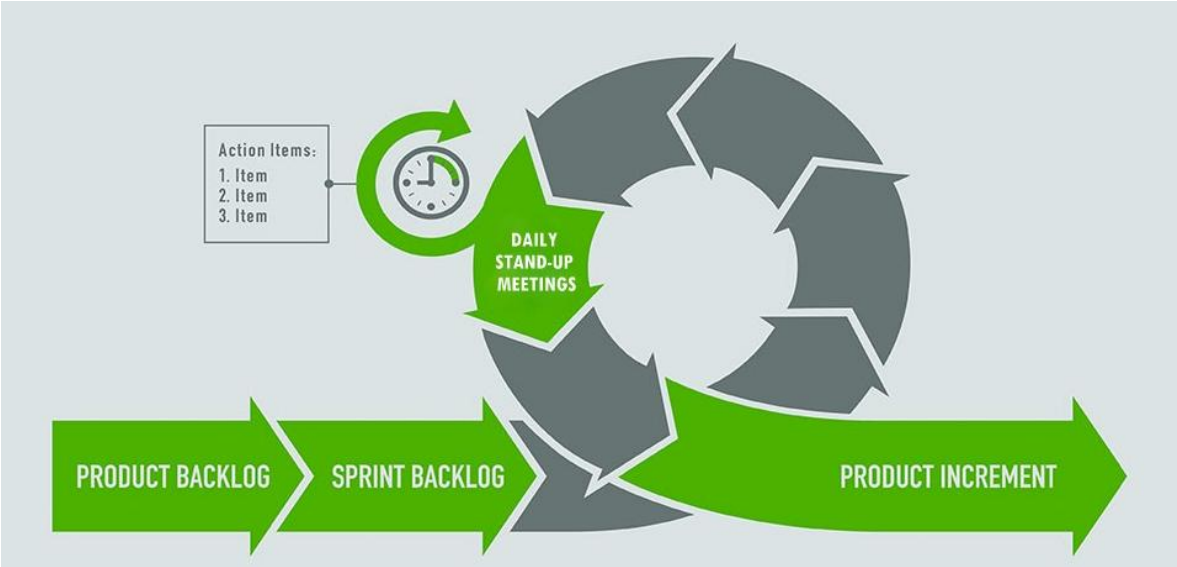


Figure 2 Agile Methodology

Agile methodology is a modern approach to project management that prioritizes flexibility, collaboration, and incremental development. It was used as the development framework in this research because of its organized yet adaptive way of handling complex and iterative work, a good match for projects requiring semantic segmentation and reinforcement learning. Scrum develops work in short, time-boxed iterations, each of which delivers a functional increment of the system. The sprint-based structure is well suited to the experimental nature of machine learning, where models tend to need to be repeatedly trained, tested, and fine-tuned.

In this research, DeepLabV3+, reinforcement learning agents, and custom Super Mario Bros. environments were implemented and optimized over numerous iterations. The Scrum process enabled frequent feedback and enabled the development team to respond immediately to dynamic results from training sessions. Through the prioritization of sprint

planning, daily stand-ups, and sprint reviews, the team was able to work consistently with incremental improvement in model performance and in efficiency of the model.

a. **Product Backlog**

The product backlog serves as a list of prioritized task that is crucial for completing the study's objectives and this backlog served as a roadmap for the integration of DeepLabV3+ segmentation model to the reinforcement learning pipeline in the 2D game Super Mario Bros. Additionally, the preparation of each processes also includes the creation of synthetic dataset and the customized game levels of the Super Mario Bros game in the SMB Utility platform. By sorting these tasks in one place, the team will be able to proceed perfectly and will also be to adapt quickly evolving requirements ensuring that all of the sprints are performed to achieve the goal of enhancing the agent's training time.

Table 7 Backlog of the project

Backlog	Priority	Estimated(Hours)
Benchmark and compare DeeplabV3+ with other semantic segmentation models.	HIGH	1
Collect & organize sprites and tilesets from Super Mario Bros.	HIGH	14
Preprocess images	HIGH	22
Configure dataset generator to assemble sprites into game-like frames	HIGH	33
Generate synthetic dataset for model training	HIGH	6
Set up a Conda environment (with PyTorch, torchvision, etc.)	MEDIUM	13

Debug training pipeline	HIGH	28
Test if the training works as expected	HIGH	20
Train DeepLabV3 model on the synthetic dataset	HIGH	40
Modify codebase to support DeepLabV3+	HIGH	30
Integrate third-party implementation of DeepLabV3+	HIGH	35
Test if model runs, trains, and saves successfully	HIGH	15
Train DeepLabV3+ model on the same dataset	MEDIUM	40
Create the layout for custom Super Mario Bros levels in SMB Utility	HIGH	25
Integrate new levels into the game's dependencies	HIGH	20
Configure RL environment	HIGH	25
Debug any errors in training code	MEDIUM	13
Train RL agent using DeepLabV3 segmented inputs and custom levels	HIGH	48
Modify RL environment from Sprint 5	MEDIUM	15
Adapt RL code to accept DeepLabV3+ output	MEDIUM	20
Train RL agent with DeepLabV3+ and custom level	MEDIUM	48
Test **Mean Intersection over Union (mIoU)** for both models	HIGH	26
Test **Mean Average Precision (mAP)**	HIGH	33

for segmentation quality Compare DeepLabV3 vs DeepLabV3+ with test metrics and gameplay footage	HIGH	26
Create Likert-scale questionnaire Prepare evaluation package (footage, graphs, logs, questionnaire) Distribute to IT experts and collect responses	MEDIUM	19
	HIGH	25
	MEDIUM	20

Table 7 includes a roadmap for a more detailed tasks and it also includes the estimated hours that will require for each tasks to finish. This backlog also includes sprints that can last up to 1-2 weeks and depending on the tasks on each sprint it may last up to 2-3 weeks time. As for the shorter sprints that require a shorter timeframe, an 80 hours of work is allocated on each tasks, while the longer sprints has 120 hours allocated to it with the assumption of 8-hour workday time. This detailed planning helped us focus on tasks that will require the most attention or has the highest priority and also be able to adjust to sudden changes along the process of the project.

b. Sprint Planning

During the sprint planning phase, detailed tasks was integrated for both the DeepLabV3 and its improved counterpart which is the DeepLabV3+, both integrated in to the reinforcement learning pipeline with the same conditions for comparison evaluation. By having both of the segmentation model have the same synthetic dataset and the customized Super Mario Bros. levels, An evaluation framework was used to assess segmentation outputs from both the models. Using the published benchmarks from previous models, the team focused on the model that offers features that will be most

beneficial for 2D platformer environment. Unlike the use of a formal experiment, the team relied on established literature to ensure that each sprint will proceed according and based on existing research that is aligned with the study's objectives.

Table 8 Sprint planning

Sprint	Focus Areas	Tasks
1	Benchmarking semantic segmentation models and generating synthetic dataset.	5
2	Setting up the environment and training the DeeplabV3 semantic model.	4
3	Implementation and training of deeplabV3+.	4
4	Designing and creating custom Super Mario Bros levels.	2
5	Training of RL agent using the trained DeeplabV3 model.	3
6	Training of RL agent using the trained DeeplabV3+ model.	3
7	Testing the model using quantitative assessment (mIoU and mAP)	3
8	Qualitative assessment of the model through expert validation using a Likert Scale questionnaire.	3

c. Sprint Backlog

Table 9 Sprint 1

SPRINT 1			
ID	TASK	DURATION(Hours)	Sory Points
1.1	Benchmark and compare DeeplabV3+ with other semantic segmentation models.	1	1
1.2	Collect & organize sprites and tilesets from Super Mario Bros	14	2
1.3	Preprocess images	22	2
1.4	Configure dataset generator to assemble sprites into game-like frames	33	3
1.5	Generate synthetic dataset for model training	6	1
TOTAL HOURS:		75	

The group started by establishing the project's structure. As they gather and arrange the necessary game assets, including some of Super Mario Bros. sprites and tilesets, and then preprocessed these pictures. In order to create a synthetic dataset for the segmentation model's training, they also set up a dataset generator to combine the sprites into realistic, game-like frames.

Table 10 Sprint 2

SPRINT 2			
ID	TASK	DURATION(Hours)	Story Points
2.1	Set up a Conda environment (with PyTorch, torchvision, etc.)	13	1
2.2	Debug training pipeline	28	3
2.3	Test if the training works as expected	20	2
2.4	Train DeepLabV3 model on the synthetic dataset	40	3
TOTAL HOURS:		101	

In this sprint, The team's focus shifted to setting up the development environment and verifying initial training processes. The team established a Conda environment equipped with PyTorch, torchvision, and other necessary libraries. They then debugged the training pipeline, tested the training process to ensure it functioned as expected, and initiated the training of DeepLabV3 segmentation model using the synthetic dataset.

Table 11 Sprint 3

SPRINT 3			
ID	TASK	DURATION(Hours)	Story Points
3.1	Modify codebase to support DeepLabV3+	30	3
3.2	Integrate third-party implementation of DeepLabV3+	35	3
3.3	Test if model runs, trains, and saves successfully	15	1
3.4	Train DeepLabV3+ model on the same dataset	40	3
TOTAL HOURS:		120	

In this sprint, the project moved towards implementing the more advanced DeepLabV3+ model. The team modified the existing codebase to support DeepLabV3+, integrated a third-party implementation of the model, and rigorously tested to confirm that it ran, trained, and saved properly. Following successful tests, they proceeded to train the DeepLabV3+ model on the same synthetic dataset.

Table 12 Sprint 4

SPRINT 4			
ID	TASK	DURATION(Hours)	Story Points
4.1	Create the layout for custom Super Mario Bros levels in SMB Utility	25	2
4.2	Integrate new levels into the game's dependencies	20	2
TOTAL HOURS:		45	

The development process began with creating the layout of the levels to be used as the training environments. Leveraging the SMB Utility software, the team customized these levels to align with the evolving segmentation model outputs. By tailoring the levels in the original game, the researchers ensured that the synthetic data and actual game environment were closely matched, thus creating a more realistic training and testing ground for the model.

Table 13 Sprint 5

SPRINT 5			
ID	TASK	DURATION(Hours)	Story Points
5.1	Configure RL environment	25	2
5.2	Debug any errors in training code	13	2
5.3	Train RL agent using DeepLabV3 segmented inputs and custom levels	48	3
TOTAL HOURS:		86	

During this sprint, the team focused on the reinforcement learning component. Debugging of any errors were performed to ensure that the training of the agent could execute without any problems, thus the developers will then start training the agent.

Table 14 Sprint 6

SPRINT 6			
ID	TASK	DURATION(Hours)	Story Points
6.1	Modify RL environment from Sprint 5	15	2
6.2	Adapt RL code to accept DeepLabV3+ output	20	2
6.3	Train RL agent with DeepLabV3+ and custom level	48	3
TOTAL HOURS:		83	

The main goal of this sprint was to improve and polish the RL environment. The team changed the RL code so that it could take outputs from the new DeepLabV3+ model. They then retrained the RL agent using these better segmentation inputs and the new custom Super Mario Bros. levels, which made the whole training process better.

Table 15 Sprint 7

SPRINT 7			
ID	TASK	DURATION(Hours)	Story Points
7.1	Test **Mean Intersection over Union (mIoU)** for both models	26	2
7.2	Test **Mean Average Precision (mAP)** for segmentation quality	33	2
7.3	Compare DeepLabV3 vs DeepLabV3+ with test metrics and gameplay footage	26	2
TOTAL HOURS:		85	

The team did tests to find out important performance metrics like Mean Intersection over Union (mIoU) and Mean Average Precision (mAP). They used both quantitative

metrics and qualitative assessments from gameplay footage to compare how well the DeepLabV3 and DeepLabV3+ models worked.

Table 16 Sprint 8

SPRINT 8			
ID	TASK	DURATION(Hours)	Story Points
8.1	Create Likert-scale questionnaire	19	1
8.2	Prepare evaluation package (footage, graphs, logs, questionnaire)	25	2
8.3	Distribute to IT experts and collect responses	20	1
TOTAL HOURS:		64	

The last sprint was all about finishing the evaluation process and getting feedback from experts. The team made a Likert-scale questionnaire, put together an evaluation package with gameplay footage, performance graphs, logs, and the questionnaire itself, and then sent these materials to IT experts to look over. After collecting responses, they analyzed the feedback to identify areas for potential design and performance improvements, marking the final step in the project's development cycle.

d. Gantt Chart

A Gantt chart is a project management tool that visually maps out tasks, deadlines, and dependencies over the project's timeline. In this study, the Gantt chart organized each phase from gathering sprites and generating a synthetic dataset, to customizing Super Mario Bros. levels via SMB Utility, to training DeepLabV3/DeepLabV3+ models and

integrating reinforcement learning. By clearly illustrating when and how these tasks overlapped, the Gantt chart allowed the team to identify critical milestones (e.g., dataset completion, model deployment, RL testing) and manage resources efficiently. This structured overview was crucial for anticipating challenges, coordinating sprints, and ensuring that all development and evaluation goals were met within the designated time frame.

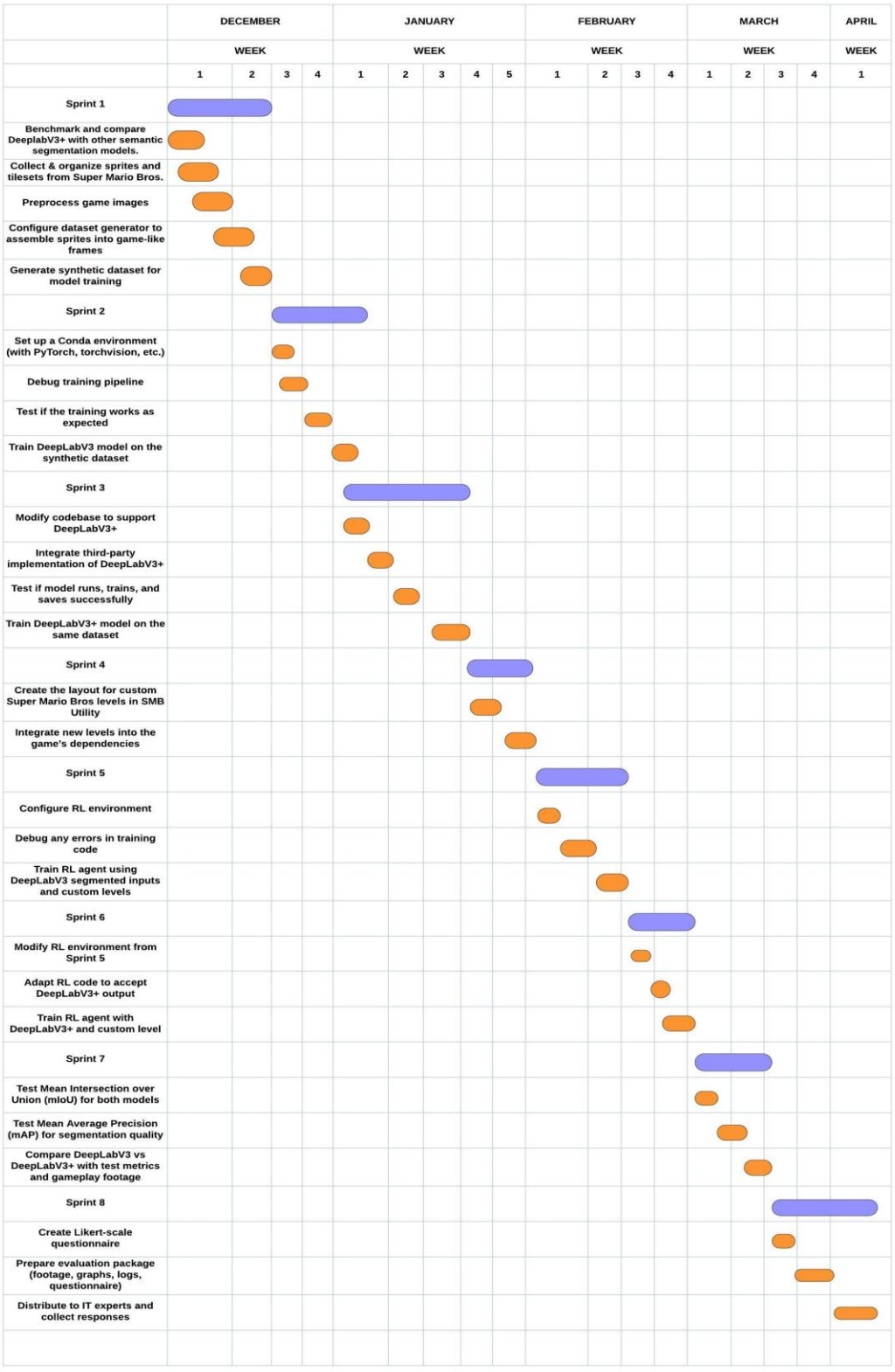


Figure 3 Gantt Chart

e. Burndown Chart

A burndown chart is a visual tool that tracks the amount of work remaining over time, providing a clear overview of progress during each sprint. It is useful for monitoring task completion, overall progress tracking, and ensuring that the development stays on schedule. By creating a burndown chart for this project, the developers were able to efficiently handle tasks with ease and on schedule.

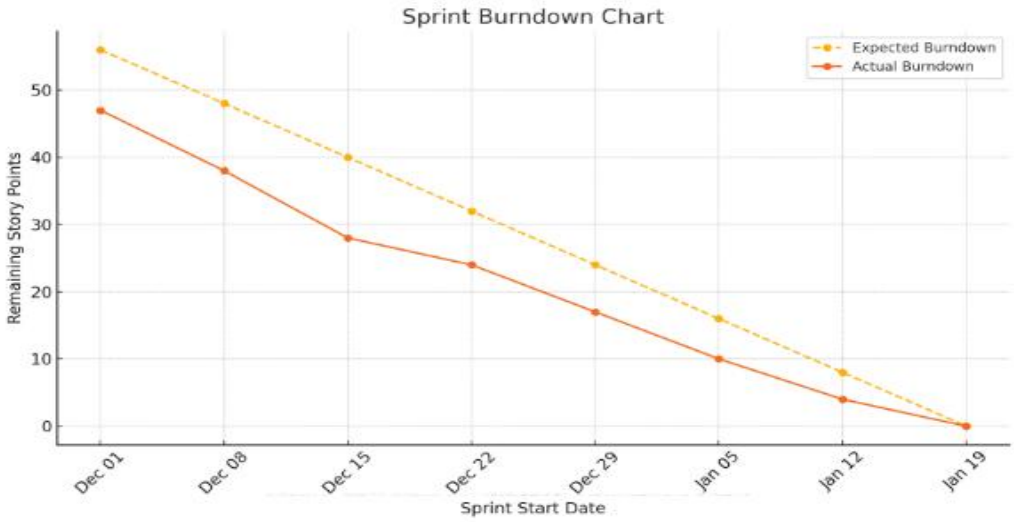


Figure 4 Burndown Chart

B. Diagrams

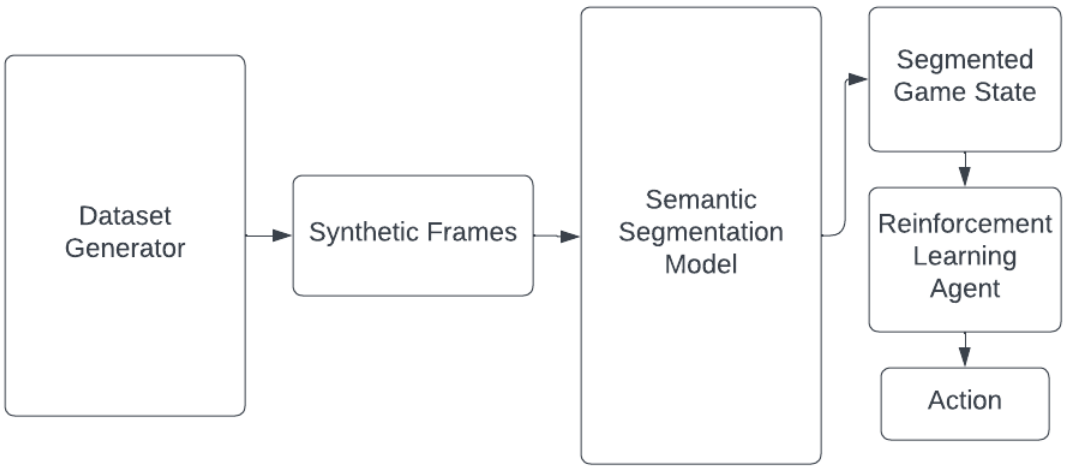


Figure 5 System Architecture

The figure above illustrates the system architecture for this study and it is divided into three primary components. The first part consists of the dataset generator, this component is responsible for generating synthetic dataset to be used in the training of the semantic segmentation models. Once the segmentation model is trained, it provides segmented frames to the RL agent, enabling it to recognize distinct game elements and interact with its environment. With the aid of the segmentation model, the RL agent uses the segmented frames sent over by the segmentation model to perform tasks and navigate the game environment towards the end goal, which is clearing the level.

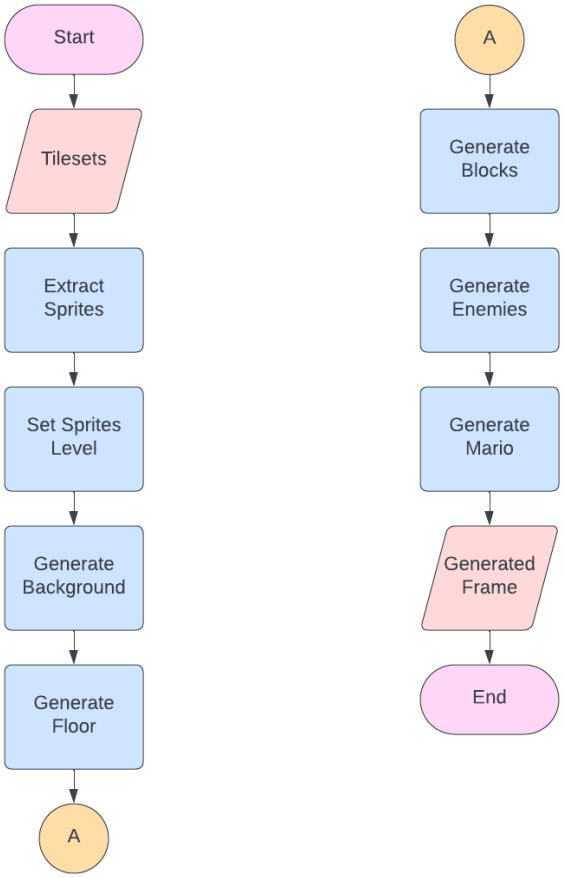


Figure 6 Dataset Generator

In semantic segmentation, a dataset goes through a rigorous process of image labeling which is highly time intensive and is done manually by the developers. To save time, the researchers used a dataset generator that produced a synthetic dataset using the Super Mario Bros' sprites. Sprites are cutouts of different elements of a video game such as blocks, enemies, and playable characters. Using this as input ensures that the domain gap between the real and synthetic frames is minimal which means that the performance difference between the two frames is reduced [5].

Figure 6 illustrates the process by which the dataset generator creates synthetic frames for the game environment that closely resemble scenes from Super Mario Bros. At

the start, a series of images containing the extracted sprites from the Super Mario Bros game, these images are known as tilesets. They are used as foundational blocks by the generator to create synthetic frames that mirror the ground truth in similarity.

Once the tilesets have been prepared, the generator extracts the sprites needed for the frame and assigns the correct layer for each sprite. This step is important as it ensures the quality of the generated synthetic frame, if the sprites are placed in the wrong layer then the generated frames cannot be used as training input for the semantic segmentation model.

After assigning the layers to the sprite, the process moves on to constructing the scene. The generator adds each element in sequence and it starts with the background sprites to create the background of the scene. Then, it generates the floor, followed by the brick blocks and other sprites classified as part of the environment. Next, the enemies are added to the scene and then the character Mario is added last; thus completing the scene. This sequential layering process ensured that all elements are positioned and rendered as they would appear in the original Super Mario Bros game, creating synthetic frames that mimicked the game's look and feel for training and testing purposes.

C. Algorithm

Method	mIOU
Adelaide_VeryDeep_FCN_VOC [85]	79.1
LRR_4x_ResNet-CRF [25]	79.3
DeepLabv2-CRF [11]	79.7
CentraleSupelec Deep G-CRF [8]	80.2
HikSeg_COCO [80]	81.4
SegModel [75]	81.8
Deep Layer Cascade (LC) [52]	82.7
TuSimple [84]	83.1
Large_Kernel_Matters [68]	83.6
Multipath-RefineNet [54]	84.2
ResNet-38_MS_COCO [86]	84.9
PSPNet [95]	85.4
IDW-CNN [83]	86.3
CASIA_IVA_SDN [23]	86.6
DIS [61]	86.8
DeepLabv3	85.7
DeepLabv3-JFT	86.9

Method	mIOU
Deep Layer Cascade (LC) [82]	82.7
TuSimple [77]	83.1
Large_Kernel_Matters [60]	83.6
Multipath-RefineNet [58]	84.2
ResNet-38_MS_COCO [83]	84.9
PSPNet [24]	85.4
IDW-CNN [84]	86.3
CASIA_IVA_SDN [63]	86.6
DIS [85]	86.8
DeepLabv3 [23]	85.7
DeepLabv3-JFT [23]	86.9
DeepLabv3+ (Xception)	87.8
DeepLabv3+ (Xception-JFT)	89.0

Figure 7 Performance of models on PASCAL VOC test set.

DeepLabv3+ has demonstrated superior performance compared to other segmentation methods, including DeepLabv3. The model achieved a mean Intersection over Union (mIOU) score of 87.8% using Xception as the backbone and 89.0% when pretrained with the JFT dataset. In experiments on the Cityscapes dataset, which includes high-quality pixel-level annotations of 5,000 finely annotated images and around 20,000 coarsely annotated images, DeepLabv3+ further proved its capabilities. When trained on the “train_fine” set, the model, using an Xception backbone with additional layers and a modified ASPP block (removing the “Image Pooling” module), achieved an mIOU of 79.55% on the validation set. To compete with other state-of-the-art models, the authors fine-tuned the best model variant using coarse annotations, resulting in a significant improvement, with an mIOU of 82.1% on the Cityscapes test set. These results highlight

DeepLabv3+'s advancement over its predecessor, DeepLabv3, and its effectiveness in high-quality image segmentation tasks.

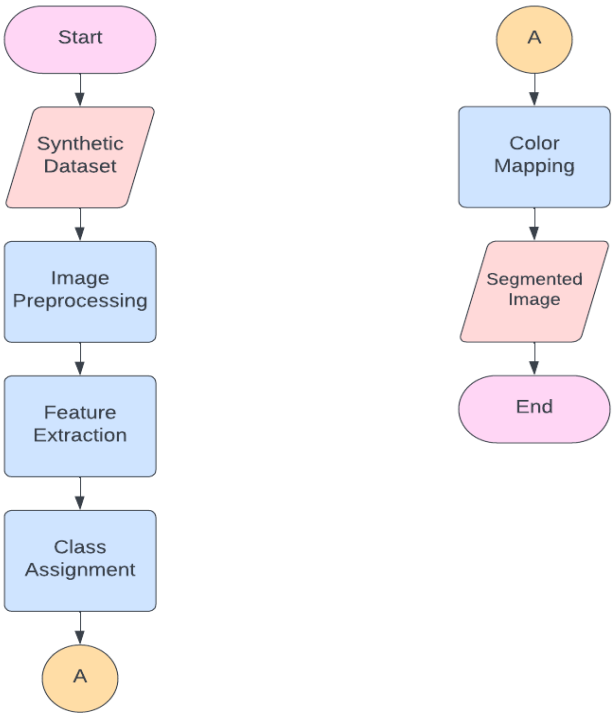


Figure 8 Segmentation Model

The flowchart indicates the training process for the segmentation model, where the synthetic dataset serves as the input, and each image is processed into a segmented output. This process begins by preparing the dataset: frames are synthetically generated from Super Mario Bros. sprites, cropped, undergo a random horizontal flip to enhance model generalization, and are converted into tensor format as **240x256 pixel grayscale** images ensuring consistent resolution and simplifying class boundaries while reducing computational overhead. Following this preprocessing, the model extracts pixel-level **pattern** and **edge** features identifying textures and contours of platforms, enemies, and characters and assigns each pixel a specific class, such as enemy, blocks, floor, or Mario.

These classes guide the model's understanding of the scene by indicating the category of each element. Once classified, each pixel is encoded as a distinct grayscale intensity through color mapping, which ensures that the segmented output visually represents different game elements with consistent monochrome "color" values, aiding in clear identification and model training accuracy.

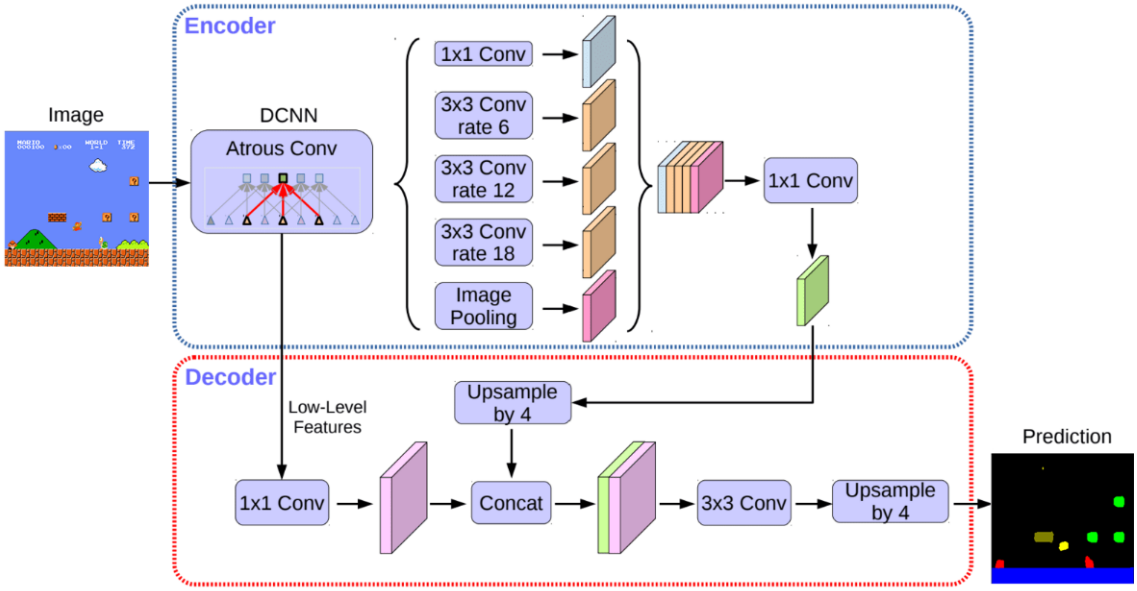


Figure 9 DeeplabV3+ Model Architecture

The DeepLabV3+ architecture follows a structured flow that begins with the encoder, moves through the ASPP module for multi-scale feature extraction, and concludes with the decoder for refined segmentation. First, an input image is sent to the encoder, a deep convolutional neural network (DCNN) that is meant to capture different levels of features. Inside the encoder, atrous convolutions are used. This setup lets the network find objects of different sizes in the image without changing the quality of the image.

After the high-level feature of the image has been found, it is sent to the decoder module to improve the segmentation map.

This module takes the low and high level features from the ASPP module and combines them. This process enhances the quality of the segmentation map and recovers the finer details lost in the high level decoding. The combination of features are then processed to match the original image size and is converted into a segmented output. The entire process enables DeeplabV3+ to create an accurate segmentation map with enhanced edge clarity and spatial consistency

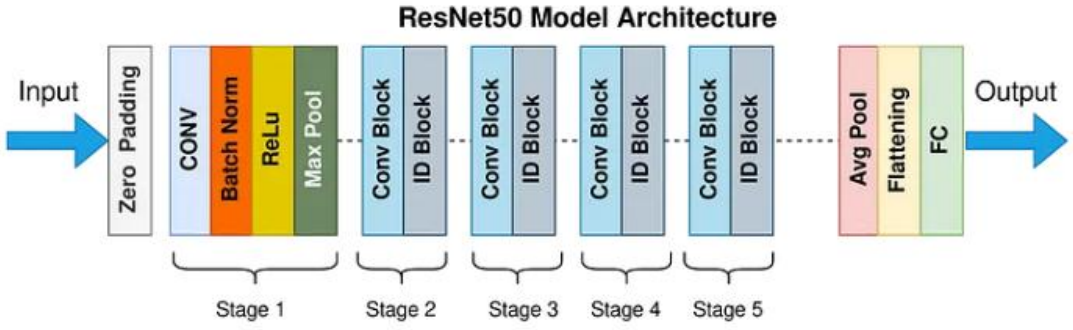


Figure 10 ResNet50 Model Architecture

ResNet50 is a deep convolutional neural network engineered to solve the vanishing gradient problem, which happens when training highly advanced networks. It uses an approach which allow layers to skip connection and instead learn the difference between the input and output of specific layers.

ResNet50, as the name suggests, is composed of 50 layers and are divided into five stages. The architecture begins with the initial convolutional layer, then batch normalization, ReLU activation, and max pooling. This is followed by a series of conv block and Identity Block (ID blocks), these apply additional convolutional and batch normalization layers. Conv blocks handle shortcuts to handle changing dimensions and ID

blocks ensure the dimensions remain the same. The step to the process applies average pooling and flattening to a connected layer to generate an output.

D. Potential Ethical Considerations

The use of synthetic data is crucial for this project as it enables the researchers to save time and resources in the training process.Since synthetic data is not from real player interactions, it must be generated carefully to ensure that it reflects multiple realistic scenarios. Otherwise, the model might overfit to the synthetic data, leading to biased or ineffective behavior in actual gameplay. Transparency about the data's artificial nature and its limitations is crucial to avoid misrepresentations and ensure that the model's performance aligns with real-world expectations.

E. StoryBoard

Figure 11 Tileset and Synthetic Frame

In Figure 11, the image on the left is a small part of the game environment's tilesets. These sprites are used to generate and mimic the elements of the game such as the background, foreground, blocks, enemies, and Mario.

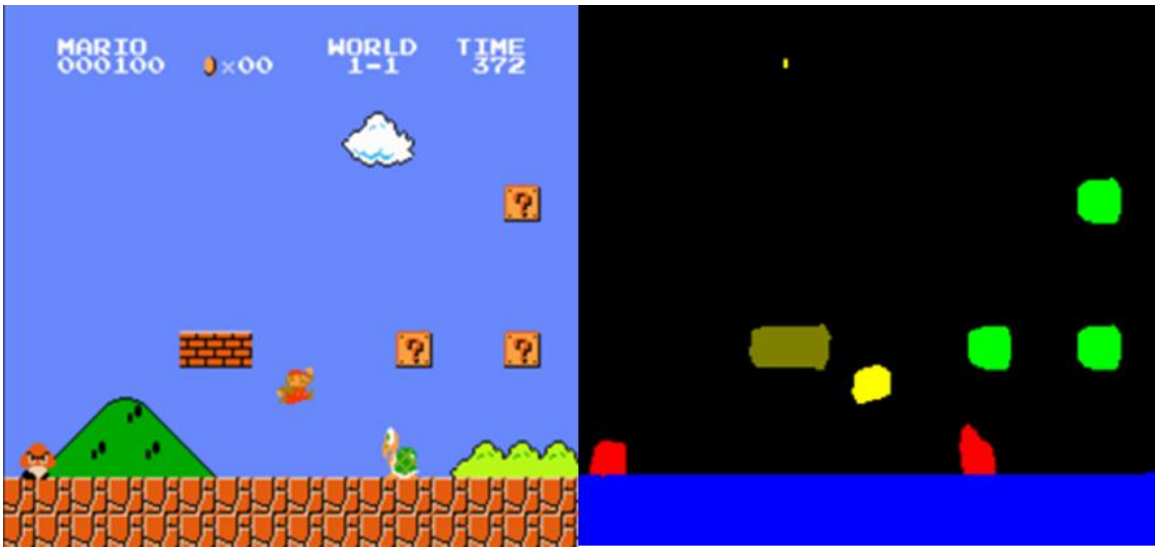


Figure 12 Synthetic frame and Segmented Version

As we can see above, the left-hand image is the synthetic frame which is created by the dataset creator and the right hand is the segmented image. The input images should be 240x256 in order to go through preprocessing and segmentation. Looking at the right-hand image, you can see different game elements are given different colours; the color shows which kind of element these silhouettes are. These are the game's frames which are extracted as its features, they are predominantly: Mario, enemies, blocks, and platform.

F. Requirement Analysis and Documentation

a. Functional Requirements

Our project aims to reduce the training time of reinforcement learning (RL) agents in 2D platformer games by leveraging DeepLabV3+ semantic segmentation. The system will generate a synthetic dataset, train segmentation models, and integrate them with an RL agent (using DQN) to navigate 2D platformer game environments, specifically the Super

Mario Bros game. This section outlines the functional requirements (what the system must do) and non-functional requirements (qualities or constraints of the system).

Table 17 Functional Requirement for Game Design

Req.ID	Requirement Description	Priority	Complexity
FR1	● The system shall generate synthetic dataset that will be used to train the segmentation models.	High	Medium
FR2	● The system shall train DeeplabV3 and DeeplabV3+ segmentation models using the generated synthetic dataset.	High	High
FR3	● The system shall allow the reinforcement learning agent to receive segmented game frames as input for decision-making.	High	High
FR4	● The reinforcement learning agent shall interact with and navigate a 2D video game environment using segmented input.	High	High
FR5	● The system shall log each episode's reward, steps taken, and success/failure status.	High	Medium
FR6	● The system shall provide real-time visualization of training progress, including episode count, reward trends, and training loss.	Medium	Medium
FR7	● The system shall periodically save model checkpoints during training for recovery and evaluation.	High	Medium
FR8	● The system shall compare training performance (time, reward, success rate) between RL agents using raw frames and those using segmented frames.	High	High

FR9	• The system shall compute mAP and mIoU to assess segmentation quality during model testing.	High	Medium

Table 17 shows the essential functional requirements characterizing the system's functionality in aid of the objective of the study: the augmentation of reinforcement learning by semantic segmentation. The requirements focus on the entire pipeline from preparing the data and training the model up to the evaluation and performance benchmarking. The requirements include generating a synthetic dataset and then training DeepLabV3 and DeepLabV3+ from this dataset. The above steps are basic, with the purpose of constructing both models under controlled situations in order to make a fair comparison through standard accuracy measures like the use of mIoU and mAP.

Closely related to this is the incorporation of segmentation output into the reinforcement learning loop. Not only must the system provide the RL agent with semantically segmented frames as input, it must also permit the agent to interact with the 2D game world in a meaningful way with this processed input. Seamlessly integrating the two is paramount for assessing the impact of segmentation on learning behaviour, specifically in the form of enhanced navigability, decision quality, and decreased training duration. Accompanying this central loop are features for training management and analysis. Among them are automated logging of episode reward and success achievement, as well as live rendering of training statistics and checkpointing for maintaining model state over extended periods of training.

Finally, a critical requirement deals with comparative performance measurement across raw and segmentation input scenarios. The functionality supports the hypothesis of the study by measuring the gain in training performance and agent performance.

b. Non-Functional Requirements

The customized Super Mario Bros game operates effectively without any major issues. Below are Non-Functional Requirements necessary to ensure the game operates without issues and guarantees for the best player experience. This will also serve as a guide on how the game will function.

Table 18 Non-Functional Requirement

Code	Dependencies Description
NF1	● The segmentation model must achieve a minimum mIoU of 0.70 and mAP of 0.75 to be considered effective for RL integration.
NF2	● The system must be able to recover from a crash or interruption by restoring the most recent model checkpoint.
NF3	● The RL agent's training process should maintain stable reward progression with no major performance drops over 10 consecutive episodes.
NF4	● mAP and mIoU must be calculated with at least two decimal places of precision to ensure evaluation accuracy
NF5	● The system must render high-resolution overlays or masks to clearly visualize class distinctions in game frames.

This table captures the non-functional requirements for the reliability and efficacy of the system in blending semantic segmentation with reinforcement learning. The requirements supplement the functional pipeline by supporting the accuracy benchmarks for segmentation models, preventing interruptions in the system from hindering the training process, keeping reward signals stable during the training process for RL, computing evaluation metrics with accuracy, and displaying segmentation outputs for analysis with

clarity. The requirements in aggregate ensure consistency, traceability, and visibility across training and evaluation.

G. Objective Testing

a. Mean Intersection over Union (mIoU)

The researchers utilized the testing method mIoU (Mean Intersection over Union) to determine the accuracy of the models in segmenting frames from Super Mario Bros. This metric is applied to evaluate how well DeepLabV3 and DeepLabV3+ distinguish various in-game elements, such as platforms, characters, obstacles, and enemies, by measuring the overlap between the predicted and ground truth segmentations [26].In order to make sure that the assessment is accurate, the researchers computed the IoU for each class and solved for the mean across all categories to determine the performance of the semantic segmentation model. The researchers also compared the performance scores of DeeplabV3 and DeeplabV3+ to determine which model provided a more accurate segmentation map.

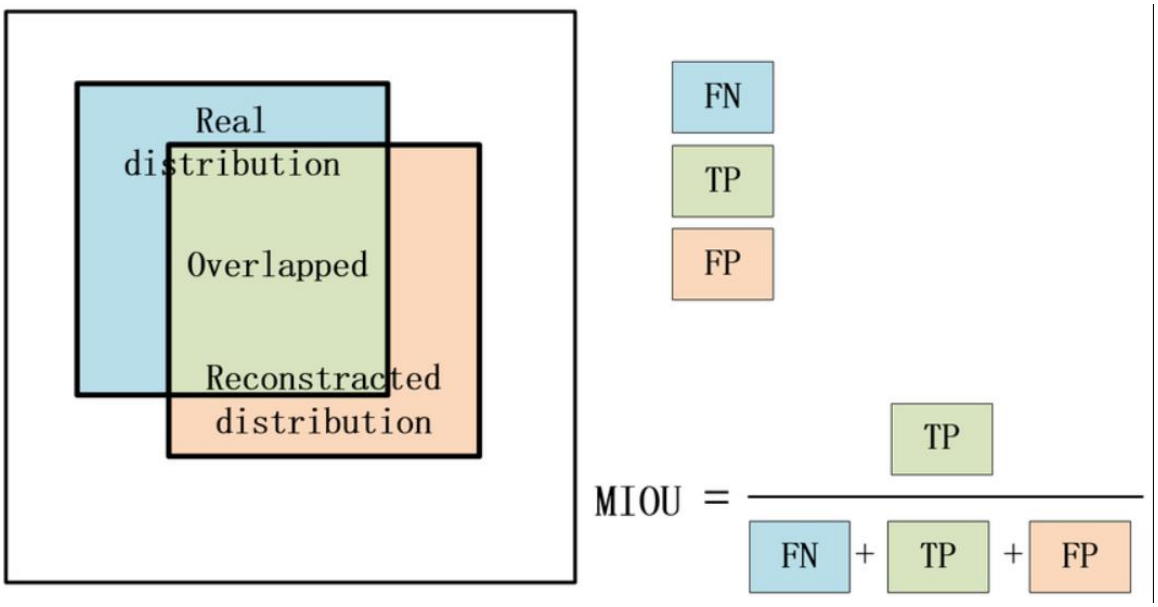


Figure 13 Diagram of Mean Intersection over Union

Mean Intersection over Union (mIoU) is a metric that evaluates semantic segmentation performance by measuring the accuracy of pixel predictions across multiple classes. The overlap between the generated segmentation masks and ground truth masks are compared and scored from 0 to 1 with 1 being the highest. Each class' IoU is calculated by dividing the area of intersection between generated and ground truth masks by their overlap; the process also includes variables such as over and under segmentation of the model. The IoU scores per class are then averaged to get the overall mIoU value and this step ensures that all classes are equally contributing to the final metric, this prevents bias from classes that may take up the majority of the pixel count.

Formula:

$$MIoU = \frac{1}{C}\sum_{i=0}^{C} \frac{TP_i}{TP_i + FP_i + FN_i}$$

Where:

- C = Total number of classes
- TP_i = True Positives for class i (Correctly predicted pixels)
- FP_i= False Positives for class i (Incorrectly predicted pixels that should not be in class i)

  FN_i = False Negatives for class i (Missed pixels that should be in class i)

b. Mean Accuracy Precision (mAP)

The researchers use mAP as the metric for evaluating the performance of the model in terms of accuracy in segmenting objects in the Super Mario Bros frames [27]. This metric was used to identify the accuracy of the model in identifying key game elements such as blocks, floors, enemies, and Mario. It is important to measure the trade off between false positives and negatives at various thresholds for each class by analyzing Precision Recall Curve and computing for the Average Precision (AP). This method tests whether the model can consistently and accurately detect objects across varying game scenes and to compare the performance of DeeplabV3 and DeeplabV3+.

$$mAP = \frac{1}{k} \sum_{i}^{k} AP_i$$

Figure 14 Formula of mAP

Mean Accuracy Precision is more applicable to use in order to assess both integrity of localization and accuracy of object detection. Since the game environment consists of multiple interactive elements, such as **platforms, enemies, coins, and power-ups**, accurately segmenting these objects is essential for reinforcement learning and gameplay automation. By using mAP, the evaluation accounts for **false positives (incorrect detections), false negatives (missed objects), and precision-recall trade-offs**, ensuring that the model not only detects but also accurately segments the key elements in the game.

Formula:

$$MaP = \frac{1}{C}\sum_{i=1}^{C} \quad AP_i$$

Where:

- C = Total number of classes

AP_i= Average Precision for class iii. It is calculated as the area under the Precision-Recall curve.

c. Evaluation

In gathering data, the instrument used in this research was a structured questionnaire. It consisted of three parts. The researchers gathered the evaluation by conducting a survey of 3 IT expert respondents. Researchers also included a link to the results of the training to evaluate. At the same time, the respondents were also requested to answer the questionnaire. Eventually, the researchers tabulated the results of the survey from each question which was able to help to analyze the study.

Statistical Treatment of Data

To determine the Quality of Segmentation, Ability of RL Agent to Clear Levels and Impact on Training Efficiency and Overall Performance, this study made use of the following statistical procedures.

Weighted Mean

It is kind of average. Instead of each data point contributing equally to the final mean, some data points contribute more "weight" than others [31]. The weighted mean is very common in statistics, especially when studying the population. The general formula is as follows.

Formula:

$$Weighted\ mean = \frac{\sum \quad (w_i * x_i)}{\sum \quad w_i}$$

Where:

w_i: weight of the i^{th} observation

x_i: value of the i^{th} observation

$\sum \quad (w_i * x_i)$: sum of the products of weights and values

$\sum \quad w_i$: sum of the weights

Steps:

1. Multiply each value by (x_i) its corresponding weight (w_i).
2. Add all these products to get $\sum \quad (w_i * x_i)$.
3. Add up all the weights $(\sum \quad w_i)$.
4. Divide the total product $(\sum \quad (w_i * x_i))$ by the total weights $(\sum \quad w_i)$.

Tally Sheets

The functionality tables show the evaluation with the help of IT experts. The findings on each table showed that all the objectives were met in terms of Interaction Capability, Usability and Security of the system.

$$\underline{x} = \frac{\Sigma_f x}{x}$$

Where: x = Weighted mean

f = Frequency of occurrence for a particular field

x= Number of units

Likert Scale

In our study, we employed a Likert scale to evaluate key aspects of the integrated DeepLabV3+ and reinforcement learning framework. Likert scales proved to be a useful tool for evaluating concepts that could not be directly observed, and the comprehensive guides available on how to develop these scales were extremely influential in the field [30]. Most studies that used survey questionnaires utilized this tool as a means of testing and examining their products based on the respondents' answers. Accordingly, our questionnaire captured expert evaluations on the quality of segmentation outputs, the performance of the reinforcement learning agent, and the overall impact on training efficiency and performance.

Table 19 Likert Scale Rating

Class Interval	Class Boundaries	Verbal Description
5	4.51 – 5.0	Strongly Agree
4	3.51 – 4.50	Agree
3	2.51 – 3.50	Neutral
2	1.51 – 2.50	Disagree
1	1.0 – 1.50	Strongly Disagree

The Likert scale is a widely used rating designed to measure IT Expert's views, opinions, or perceptions, often implemented in questionnaires and surveys. It allows respondents to express their level of satisfaction or dissatisfaction with a given statement using a numerical scale. As shown in the table, the scale categorizes the responses from Strongly agree to Strongly disagree and the mean ranges dictates which category the response belongs. This enables the researchers to come up with conclusions with regards to the respondents' answer.

Table 20 Survey Questionnaire 1

Category 1: Quality of Segmentation	1	2	3	4	5
The segmented objects' boundaries accurately align with the actual object edges in the original frame.					
All relevant objects in the original frame are fully captured in the segmented output without omissions.					
Handling of Overlapping Objects: The segmentation effectively distinguishes and separates overlapping or adjacent objects.					
The segmented output minimizes the inclusion of irrelevant background noise or artifacts.					
The segmentation enhances the contrast between objects and the background, improving visual clarity.					
The segmentation accurately handles objects of varying sizes within the frame.					

The segmented output provides a clear and accurate representation of the original frame's content.					
The edges of segmented objects are sharp. and well-defined, without blurring.					
The segmentation effectively captures and differentiates areas with varying textures within the image.					
The segmented output demonstrates a high level of confidence, with minimal ambiguity in object boundaries.					

Table 21 Survey Questionnaire 2

Category 2: Ability of RL Agent to Clear Levels	1	2	3	4	5
The agent effectively navigates around obstacles to progress through the level.					
The agent successfully identifies and avoids hazards or threats within the environment.					
The agent reaches the end goal or completes the objectives of the level					
The agent selects the most efficient paths to advance through the level.					
The agent makes timely decisions that contribute to effective level completion.					
The agent utilizes environmental features (e.g., platforms, shortcuts) to its advantage in clearing the level.					
The agent's movements and actions are precise and contribute to successful level navigation.					
The agent demonstrates effective timing and coordination in executing actions, such as jumps or attacks.					
The agent effectively responds to dynamic elements within the level, such as moving platforms or enemies.					
The agent exhibits a high level of competence in navigating and completing the level..					

Table 22 Survey Questionnaire 3

Category 3: Impact on Training Efficiency and Overall Performance	1	2	3	4	5
The graph shows a clear trend of reward improvement over time.					
There is a consistent upward trend in the average reward.					
The learning progression appears efficient and fast.					
The agent shows substantial improvement before 1500 episodes.					
The reward curve stabilizes in the later stages of training.					
The graph reflects successful retention of learned behavior.					
The reward drops are infrequent and recover quickly.					
The training is robust against instability or regressions.					
The reward curve implies that the agent has optimized its strategy.					
The learning rate of the agent can be considered fast based on the reward trend.					

In this survey questionnaire, the assessment is organized into three categories, these are:

• Category 1: **Quality of Segmentation (30%)**: This section assesses the accuracy of the models' segmentation, particularly the edge accuracy, clarity, and general visual quality. This categories' weight of 30% reflects how important it is to the researchers as part of the assessment as it has a huge impact towards the result of the project.

• Category 2: **Ability to Clear Levels (30%)**: This part focuses on the reinforcement learning agent's performance in traversing the game state and navigating towards the level's goal. The questions in this section address variables such as the agent's response to its environment, decision making, and proficiency in overcoming obstacles.

• Category 3: **Training Efficiency (40%)**: This is the most important of the three categories as it is about evaluating the efficiency of the model's training time. It has a weight of 40%, to highlight the importance of this category, as the primary goal of this study is to determine whether the use of DeeplabV3+ over DeeplabV3 would decrease the training time of the reinforcement learning agent.

The researchers will calculate the mean response for both DeeplabV3 and DeeplabV3+ models by category. These category-specific averages will then be combined, respecting the overall category weights (30%, 30%, and 40%), to produce an overall summary of the findings. This approach allows us to compare the two models comprehensively across all dimensions and provides a balanced assessment of their strengths and weaknesses before the final results are obtained.

5 RESULTS AND DISCUSSION

In this section, the researchers showed the Customized Super Mario Bros levels and explained how each level can interact with the agent or the player. Also, this features the segmented frames from these customized games using the DeepLabv3 and DeepLabv3+. Additionally, this section does not include any object label or legend in each frame since the agent can already recognize the categorization of the objects in the segmented frames.

A. To develop and train a semantic segmentation model for DeepLabV3+ to make smarter gameplay decisions using segmented output from DeepLabV3+

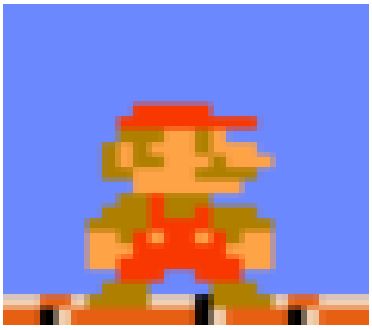


Figure 15 Mario

Figure 15 displays the avatar for the customized Super Mario Bros game, which serves as the character for both the player and the agent within the game. This avatar represents the main entity controlled by the player and the one navigated by the agent

during gameplay, providing a consistent visual representation throughout the game's progression.

Figure 16 Collectibles

The researchers also designed custom levels that featured collectibles like coins and mystery boxes. These boxes contain coins and mushrooms, which are essential for Mario to level up as he progresses through the game.

Figure 17 Enemies

As shown above, the images highlight the different enemies featured in the custom levels created by the researchers. In the top-left corner, there's Lakitu, a floating enemy that bombards Mario with obstacles throughout the stage. It's considered one of the

toughest foes to handle. Next to that is an image of Bloopers, squid-like creatures from the Super Mario series. On the top-right, we have the Koopa, often referred to as the turtle in Mario, which drops its shell upon being defeated. Mario can use this shell to take out nearby enemies, but if the shell bounces off an obstacle, it can end up harming Mario if it hits him. In the bottom-left corner, we see the Goomba, the iconic mushroom enemy that relentlessly moves toward the player. In the bottom-middle of the image, there's the Piranha Plant, which pops up when Mario gets too close to a specific pipe. Finally, the Bullet Bill, a dangerous projectile, can kill Mario if it hits him as it is launched horizontally.

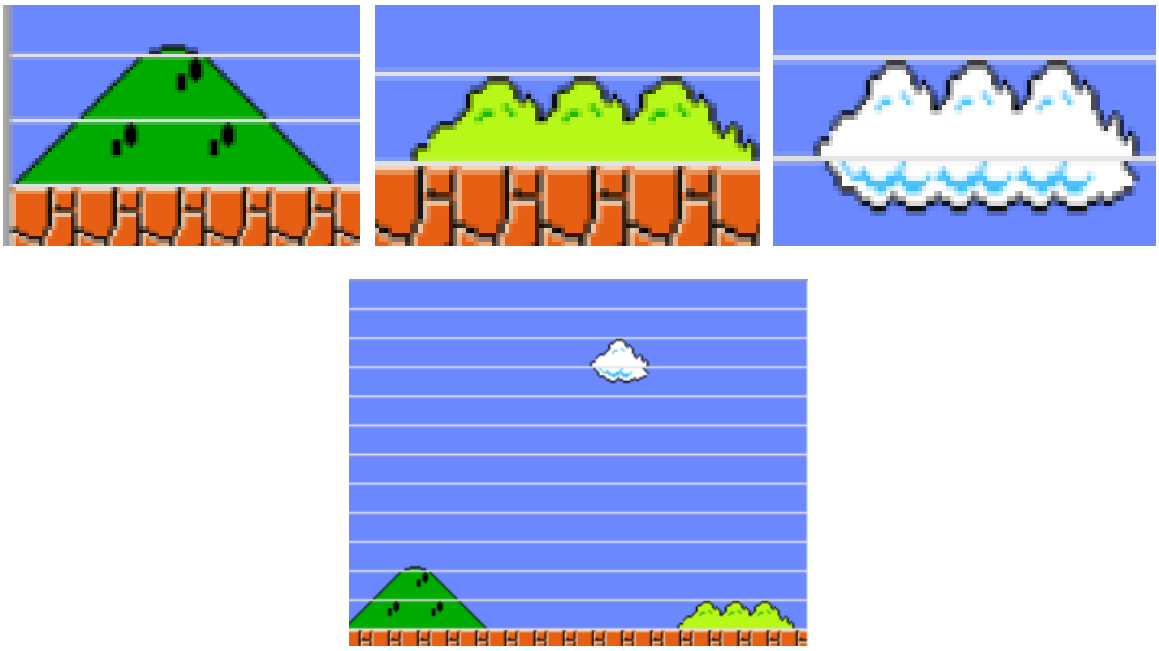


Figure 18 Game Background

These sprites serve as the game's backdrop. The background sprites—Mountains, Hills, and Clouds—don't interfere with the agent's progress, as it doesn't collide with them.

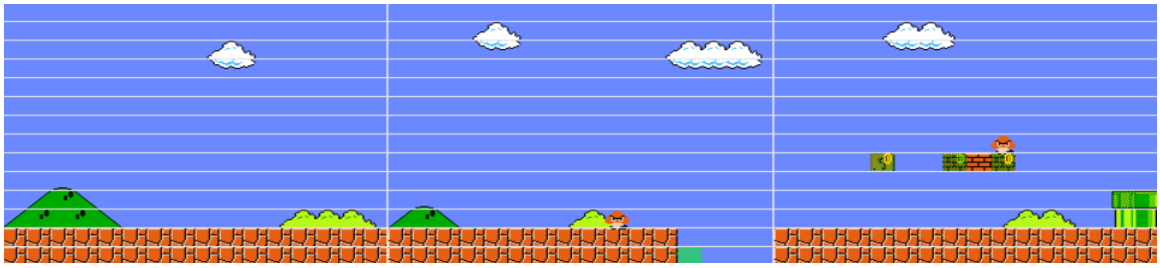


Figure 19 Level 1-1, Scene 1

This is Scene 1 of the first level, which introduces two Goombas and a hole that acts as an obstacle for the RL agent. These elements created challenges for the agent to overcome. These challenges included the enemy Goombas and the hole's potential to hinder its progress. In addition to this, mystery boxes are scattered across the scene, which contain various rewards that the agent must figure out how to activate throughout its gameplay.

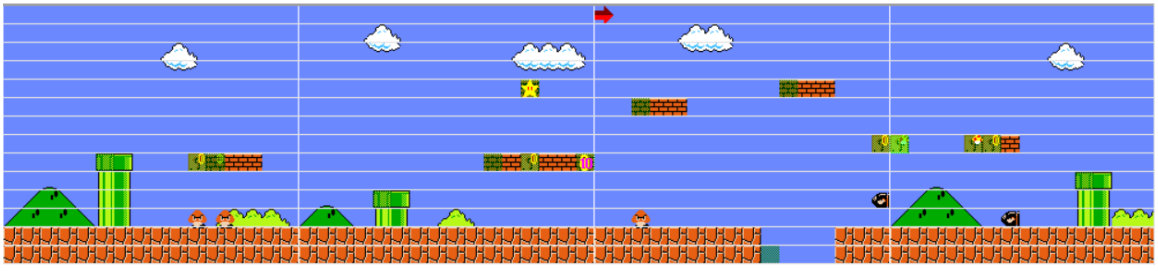


Figure 20 Level 1-1, Scene 2

Scene 2 of level 1-1, which has a more complex environment that became a challenge for the agent. This scene included large pipes, multiple enemy interactions, and a stacking platform. These new elements in the scene allowed the agent to strategize on overcoming them, such as an opportunity to avoid enemy projectiles by using the platform, and avoid the danger in order to progress. These varieties of obstacles and new elements give priority to pushing the agent to learn and adapt.

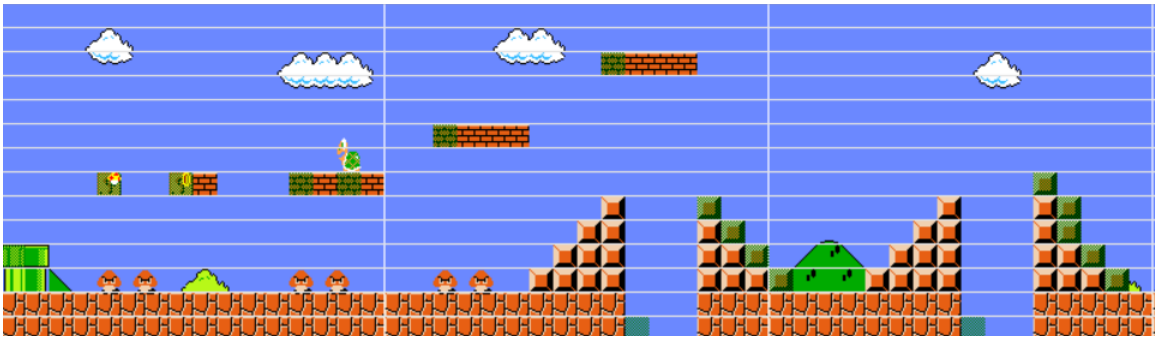


Figure 21 Level 1-1, Scene 3

This figure marks the introduction of a new enemy for the agent: the Koopa. The agent encounters the Koopa while navigating a stacking platform, adding a layer of complexity to its movement. Below the platform, multiple Goombas are scattered, providing the agent with frequent opportunities for interactions and requiring quick decision-making to avoid or defeat them. In addition to these new challenges, the scene also includes a series of stairs leading to a pitfall, further testing the agent's ability to navigate hazards. These stairs create a strategic risk, as the agent must carefully assess each step to avoid falling into the pit below. The combination of the new enemy, multiple

Goombas, and the dangerous terrain elevates the difficulty, encouraging the agent to employ more advanced tactics and enhance its problem-solving skills.

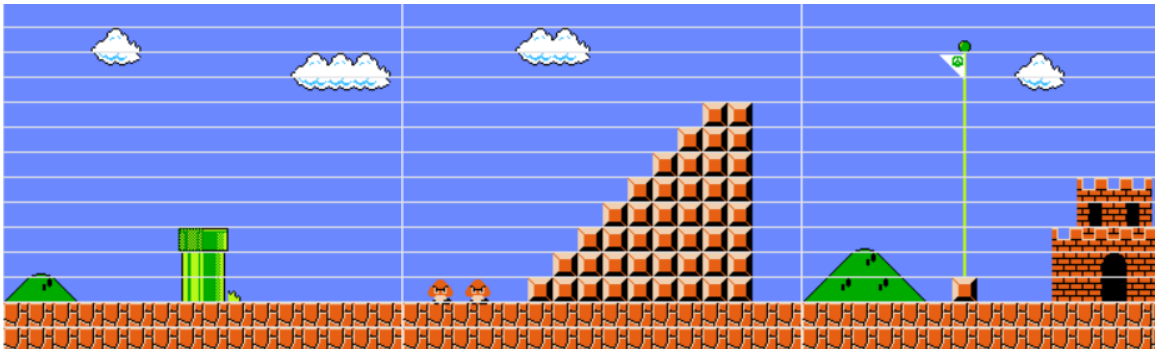


Figure 22 Level 1-1, Scene 4

This scene marked the conclusion of the stage or level, where we opted to feature the classic flagpole and castle level clearing as the final elements of this level.

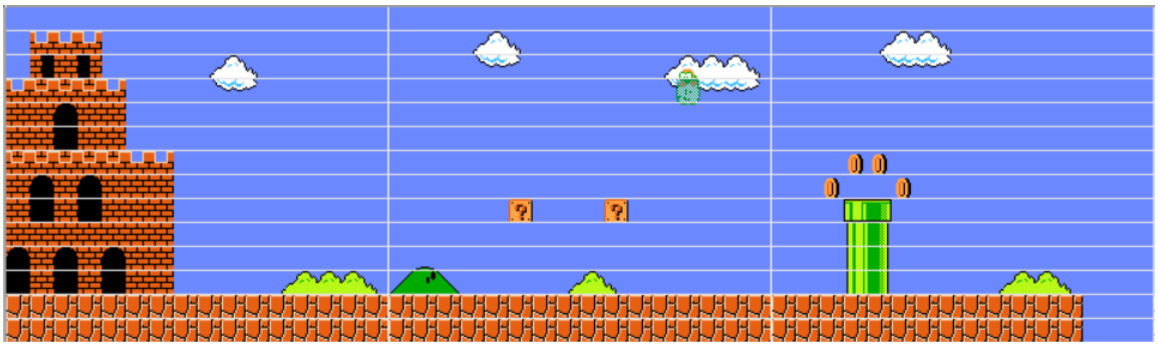


Figure 23 Level 4-1, Scene 1

This figure showcases a castle gateway that links different levels together. The castles signify that Mario has successfully completed the previous stage. In this scene, mystery boxes are placed at the start, offering rewards to the player. However, there's also an enemy lurking above, which follows Mario and drops additional enemies that pose a serious threat—touching or jumping on them will result in Mario's defeat. To add an element of surprise, a pipe containing a Piranha Plant is included, ready to catch the agent off guard. But if the agent manages to clear the pipe safely, they can collect coins for extra points, rewarding careful navigation and quick thinking.

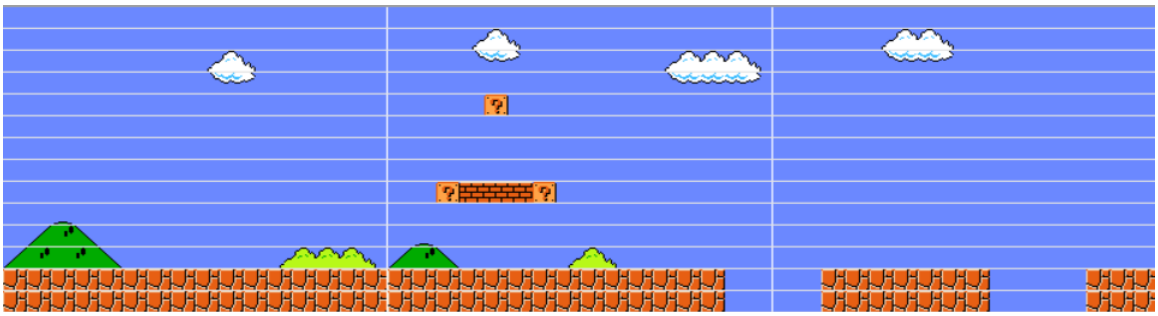


Figure 24 Level 4-1, Scene 2

The two holes served as key challenges for the agent, designed specifically to test and enhance its learning capabilities. These obstacles were strategically placed in the environment to observe how the agent adapted to different situations. By monitoring the agent's response to these holes, we assessed its ability to learn and adjust its actions in real-time. This setup provided valuable insights into the agent's progress, helping us track how effectively it navigated various hazards and improved its decision-making skills across different levels in the game. Each customized level introduced new variations of these holes, offering a more comprehensive understanding of the agent's learning curve and its ability to overcome increasingly complex obstacles.

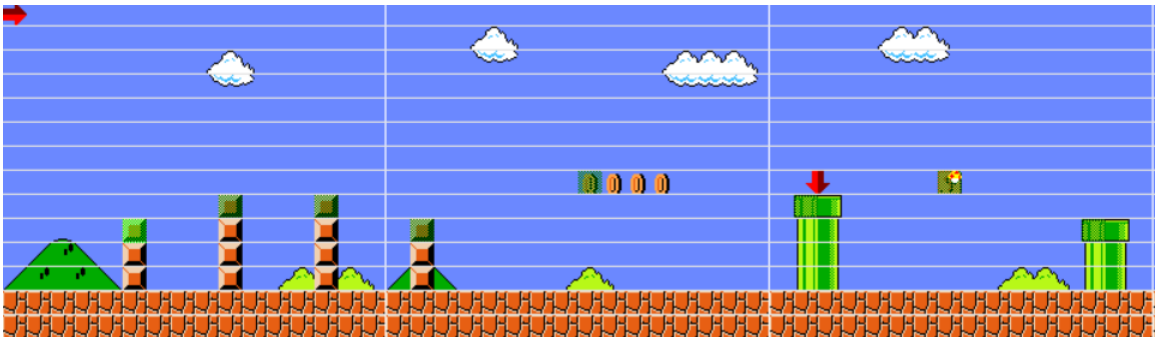


Figure 25 Level 4-1, Scene 3

The vertical blocks in this scene were instrumental in assessing the agent's jumping proficiency. These blocks were strategically placed to observe how well the agent could

time and execute jumps, a crucial aspect of navigating the game environment. By introducing these vertical challenges, we were able to track the agent's ability to adapt its jumping mechanics in response to varying heights and distances. This setup provided a clear measure of the agent's precision and coordination, helping us evaluate its progress in mastering jumping as a core skill.

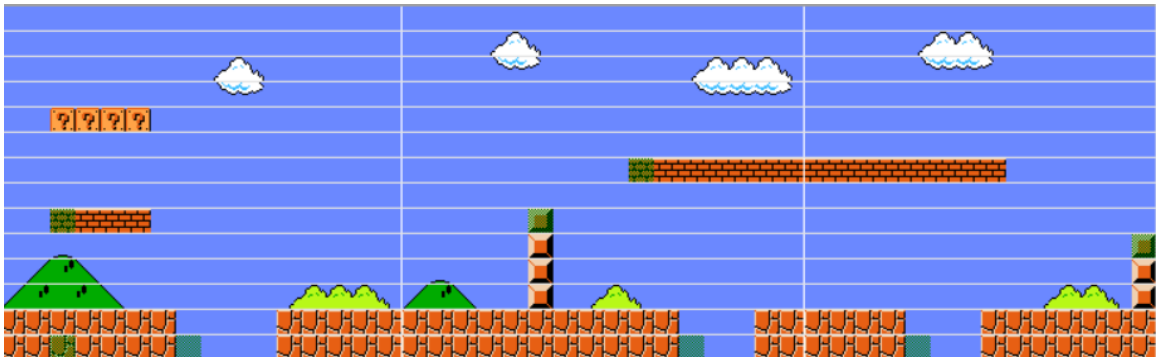


Figure 26 Level 4-1, Scene 4

The scene depicted here presents a significant challenge for the agent. It features a vertical block that must be cleared, but the platform above complicates the agent's jumping mechanics. If the agent attempts to jump too quickly, it will collide with the platform and fall into the hole below, forcing a stage reset. The combination of the vertical block, the limited jump speed, and the narrow platform creates the main obstacle in this level, requiring the agent to refine its jumping technique. Additionally, the presence of mystery boxes and other terrain elements adds further complexity, but the key hurdle remains the agent's ability to successfully move pass the platform and avoid falling.

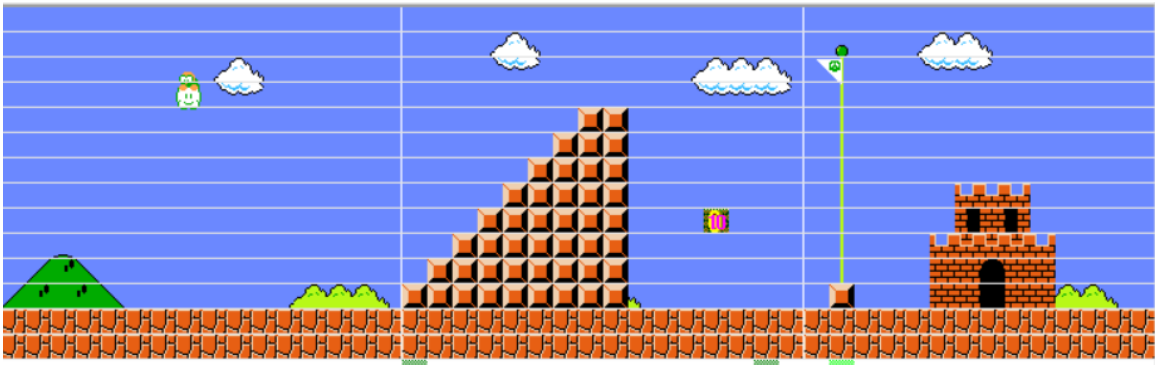


Figure 27 Level 4-1, Scene 5

In the final scene of Level 4-1, a giant staircase appeared, marking the agent's completion of the stage. The agent had to navigate this obstacle while avoiding the Lakitu enemy, who hovered above and dropped additional threats. This stairway served as both a visual and gameplay cue that the level was nearing its end. Additionally, the scene featured a flagpole, which, when reached, led the agent to a castle that acted as a gateway to the next level.

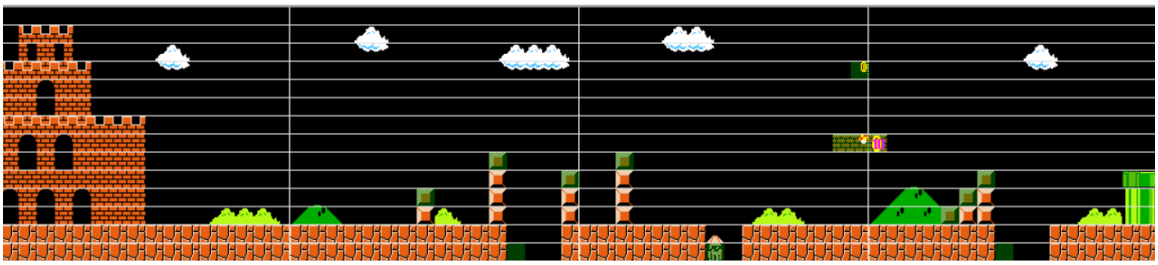


Figure 28 Level 6-1, Scene 1

The layout of Level 6-1 presents a significant challenge to the agent. The first scene introduces a combination of vertical obstacles and pitfalls that the agent must navigate with care. A vertical block forces the agent to jump at the right height to avoid colliding with it, while a hole below it creates an additional hazard. To make matters worse, an enemy is positioned in the hole, posing a lethal threat if the agent falls into it. The combination of these obstacles, including the Blooper that follows the agent throughout the level, increases

the difficulty, requiring the agent to carefully time its movements and jumps to avoid falling and being defeated.

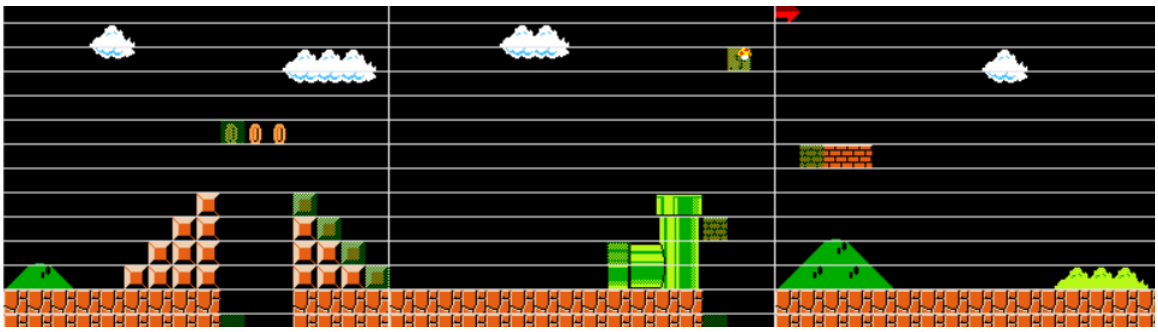


Figure 29 Level 6-1, Scene 2

In the first section, there are stacked brick blocks, which may contain items or rewards when interacted with. The next area features a raised platform with a mystery box above it, adding a challenge for the agent to figure out how to reach it. Further along, there is a green pipe that may serve as a means for the agent to travel to a different part of the level or provide another obstacle to overcome.

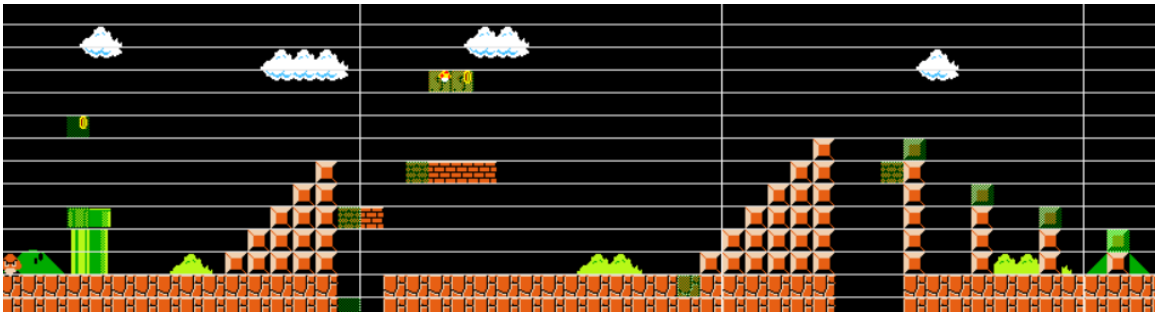


Figure 30 Level 6-1, Scene 3

In this scene, we have combined stairs with pitfalls and stacking platforms to increase the difficulty for the agent, as this is the final level it needs to clear or learn. The stairs with pitfalls present a significant challenge, requiring the agent to carefully time its movements to avoid falling, while the stacking platforms introduce an additional layer of complexity, demanding precise jumps and quick adjustments. This combination of obstacles ensures that the agent faces a tough test, pushing it to apply all the skills learned throughout the game as it navigates through the final level

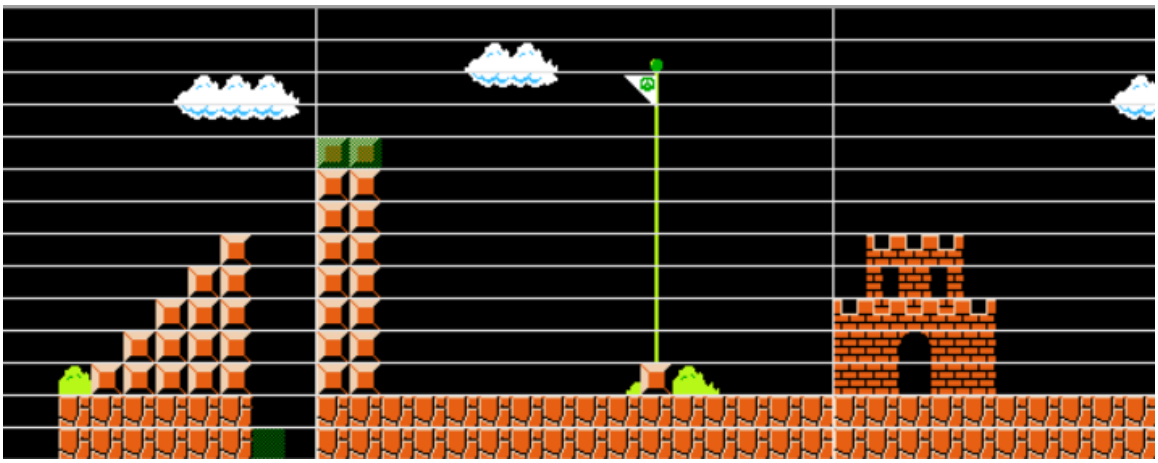


Figure 31 Level 6-1, Scene 4

This is the final scene of the customized Level 6-1. While it marks the end of the level, a significant obstacle still remains—the pitfall in the middle of the stairs. The agent must carefully navigate this hazard, as falling into the pit would result in a stage reset. The scene features a flagpole marking the end of the level, which leads to the castle that signifies the agent's successful completion of the stage. However, the lingering pitfall presents a final test of the agent's skills and ability to avoid dangers, even as it nears the end of its journey through this level. The presence of the pitfall ensures that the challenge remains until the very last moment.

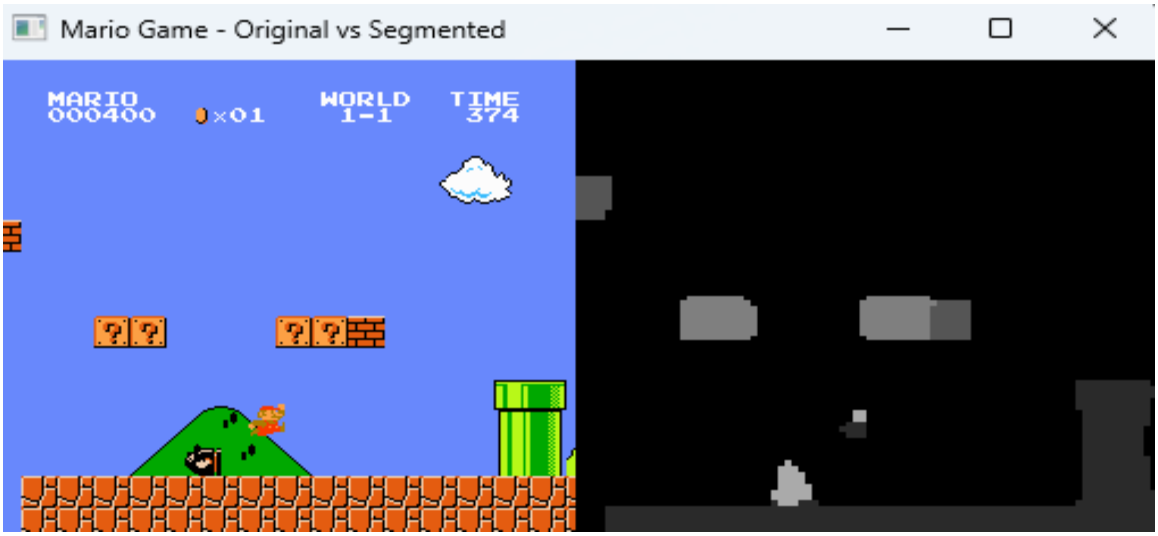


Figure 33 DeepLabV3 Segmentation in 1-1

This figure represented the segmented customized level 1-1 of Super Mario Bros for DeepLabv3. The successful segmentation of the game environment (Mario, enemies, background, and platforms) is represented in this figure. Also, the agent is successfully represented in this image.

Figure 32 DeepLabV3+ Segmentation in 1-1

This image shows the segmented customized Level 1-1 of Super Mario Bros for DeepLabV3+. It illustrates the successful segmentation of the game environment, including Mario, enemies, the background, and platforms. Additionally, the agent is clearly represented within the segmented environment, demonstrating its interaction with the game elements.

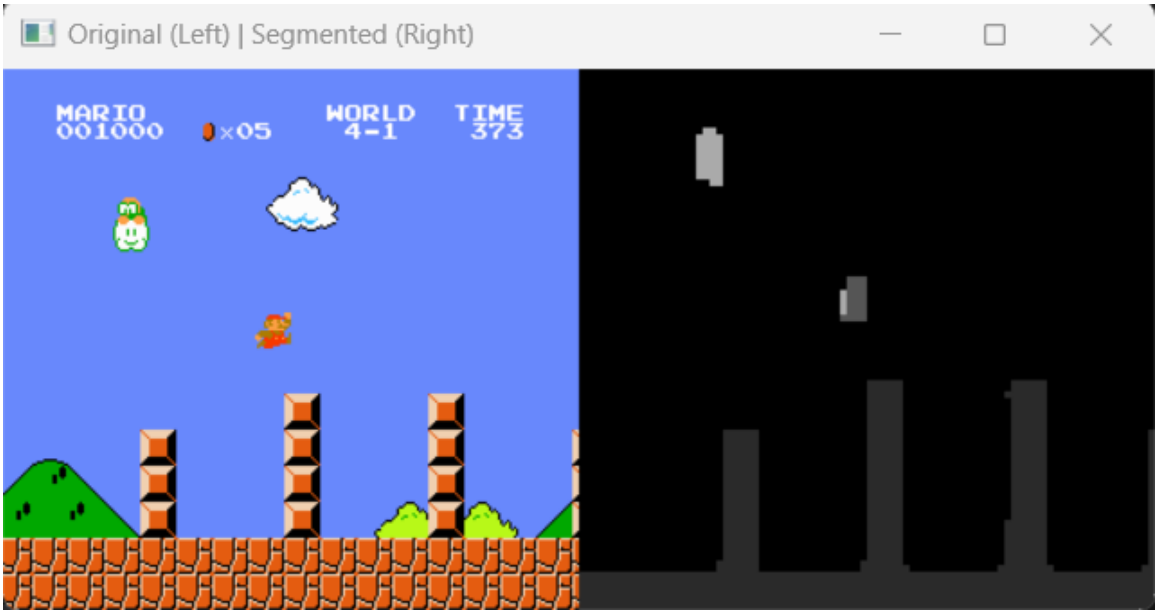


Figure 34 DeepLabV3 Segmentation in 4-1

The image depicts the segmented customized Level 4-1 of Super Mario Bros for DeepLabV3. It highlights the successful segmentation of the game environment, including Mario, enemies, the background, and platforms. The agent is also clearly represented, showing its interaction with the environment.

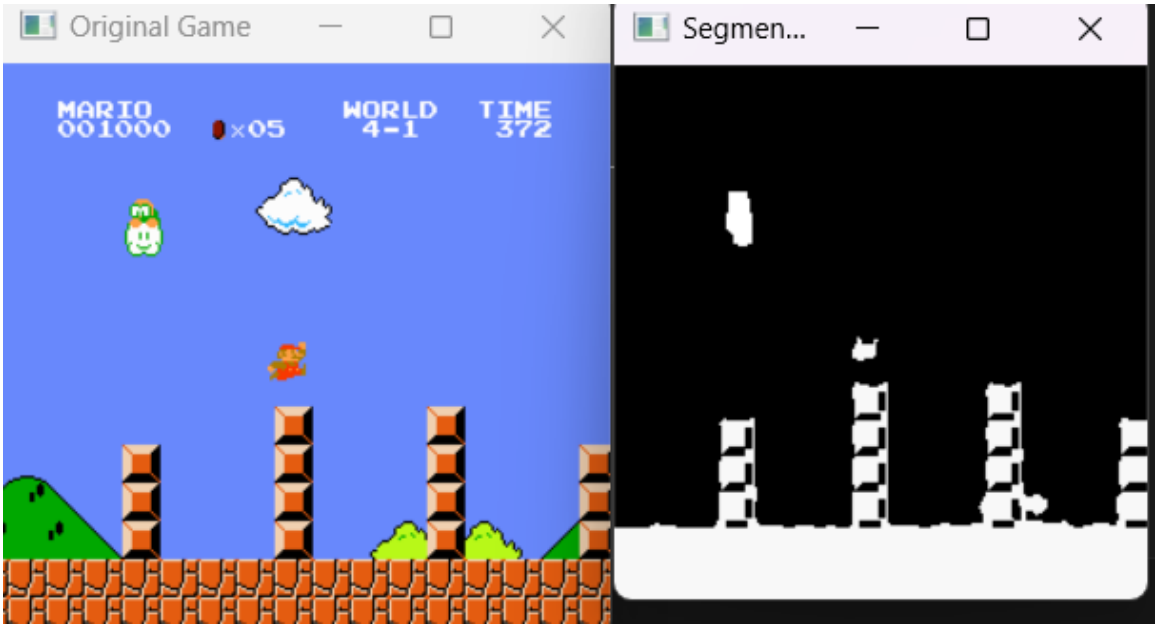


Figure 35 DeepLabV3+ Segmentation in 4-1

This frame portrays the successful segmentation of the game environment, including Mario, enemies, the background, and platforms. Additionally, the agent is clearly represented, demonstrating its interaction within the environment.

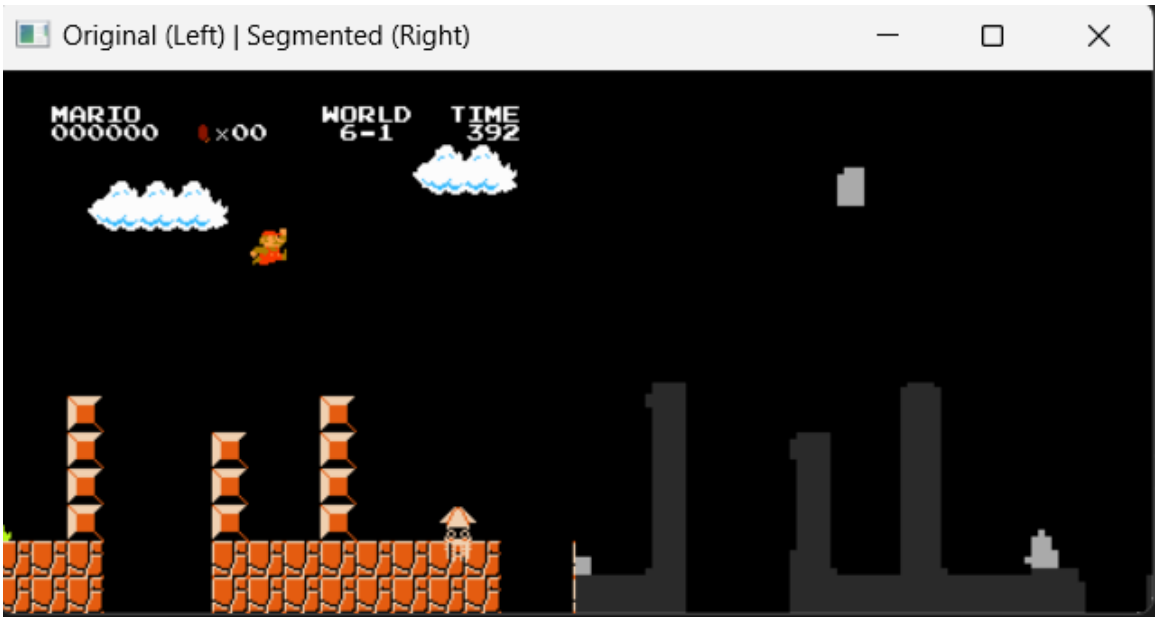


Figure 36 DeepLabV3 Segmentation in 6-1

The customized Level 6-1 of Super Mario Bros has been successfully segmented using DeepLabV3. This segmentation clearly distinguishes the various elements of the game environment, such as Mario, enemies, the background, and platforms.

Figure 37 DeepLabV3+ Segmentation in 6-1

The customized Level 6-1 of Super Mario Bros has been effectively segmented using DeepLabV3+. The segmentation clearly distinguishes various elements of the game environment, including Mario, enemies, the background, and platforms. The visual separation between these elements is evident, allowing for a precise identification of each component within the scene. This clear segmentation enables a more accurate analysis of the agent's interactions with the game environment.

Real-time labeling of game entities in the output of the segmentation was not adopted, as this is not possible with the scope of this project. Such a function would necessitate the use of more computational power that would considerably hamper the reinforcement learning process, slowing the learning speed of the agent and the overall system. Instead, the emphasis was on utilizing the fine-grained capacity of DeepLabV3+, which offered more detailed and context-enriched output as opposed to DeepLabV3. These

output maps were enough for the RL agent to parse game objects without requiring labeled overlays in the process of learning. The unsegmented and segmented frames were presented side-by-side in the evaluation for the purpose of clear visibility of the agent's behavior in relation to the original gaming landscape.

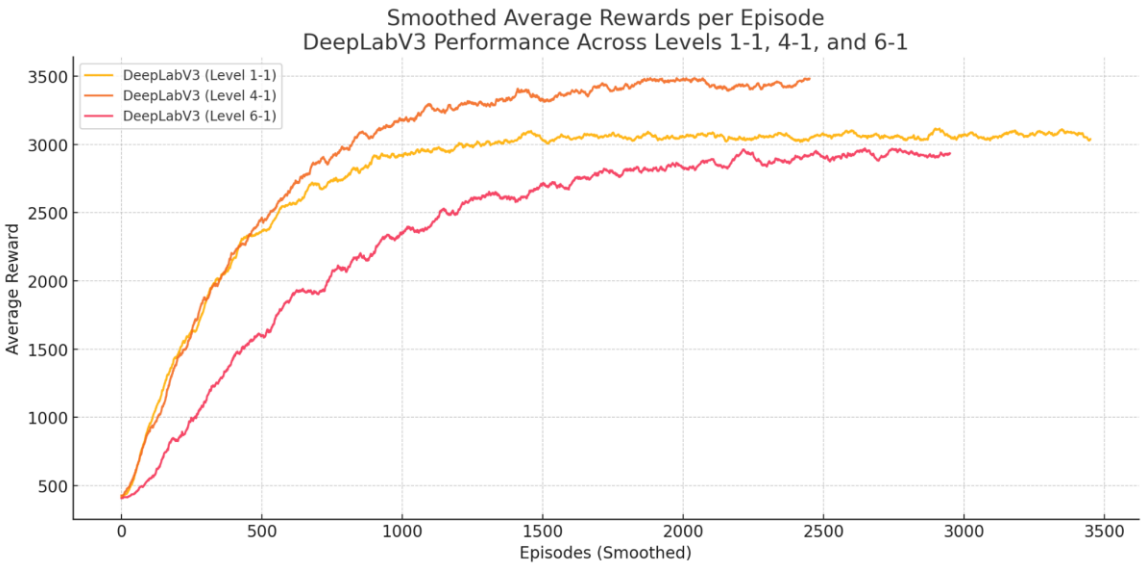


Figure 38 DeepLabV3 performance across Levels 1-1, 4-1, and 6-1

The figure illustrates the smoothed average rewards per episode for DeepLabV3 across Levels 1-1, 4-1, and 6-1, highlighting the model's performance as task complexity increases. In Level 1-1, which features a relatively simple layout and fewer obstacles, the model showed steady improvement and stabilized after approximately 1000 episodes. This indicates that DeepLabV3 is capable of learning effectively in less complex environments, allowing the agent to quickly identify patterns and optimize its behavior to achieve high rewards.

As the environment becomes more challenging in Level 4-1, with the introduction of additional enemies, vertical platforms, and more dynamic obstacles, the training curve

shows a slower progression. The model required around 1500 to 1800 episodes to reach consistent high reward levels, demonstrating that increased visual and structural complexity directly impacts the learning speed. The agent needed more time to adapt its actions in response to the segmented environment, suggesting that DeepLabV3's segmentation quality may begin to limit performance as complexity rises.

In Level 6-1, the most difficult of the three, the learning progression slowed even further. It took more than 2000 episodes for the model to approach peak performance, and even then, the curve plateaued below the maximum levels seen in the simpler stages. This level included more overlapping elements, vertical challenges, and enemies positioned in difficult locations, which posed significant segmentation and navigation difficulties. The slower learning curve reflects DeepLabV3's limitations in precisely segmenting complex scenes, which in turn affects the agent's ability to make optimal decisions.

Overall, the results suggest that while DeepLabV3 offers stable learning over time, its convergence rate is relatively slow, especially in high-complexity environments. This indicates a trade-off between model stability and training speed. As task complexity increases, so does the training time, revealing DeepLabV3's limitations in scaling efficiently across varying levels. These findings highlight the need for improved segmentation models or the integration of more advanced reinforcement learning algorithms to accelerate learning and improve performance in more demanding game scenarios.

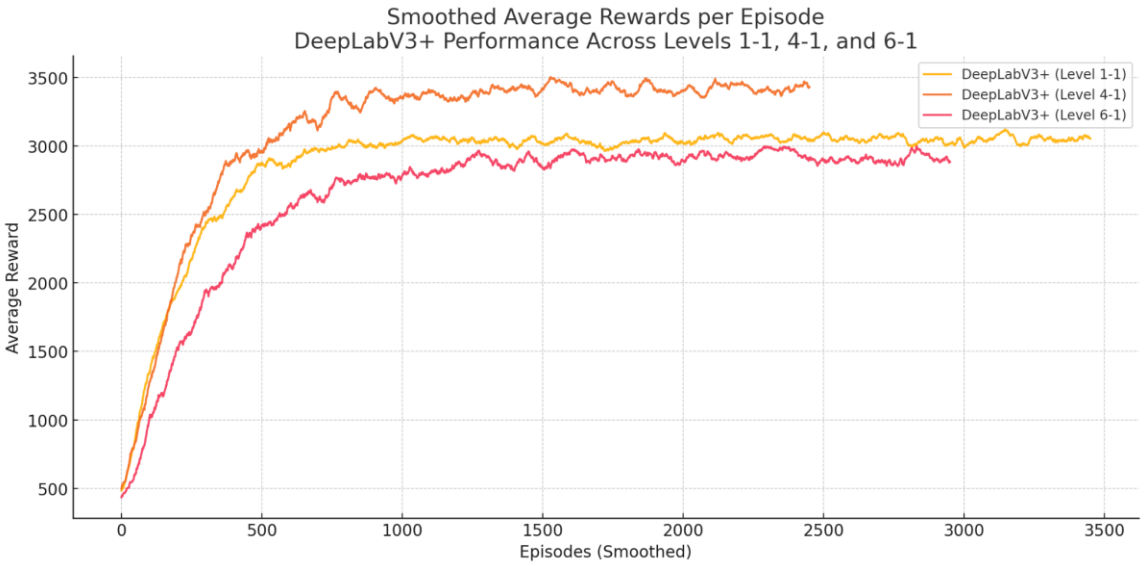


Figure 39 DeepLabV3+ performance across Levels 1-1, 4-1, and 6-1

The graph provides a clear visualization of the enhanced performance delivered by DeepLabV3+ in comparison to DeepLabV3 across three levels of increasing difficulty—1-1, 4-1, and 6-1. One of the most striking observations is the steepness of the learning curve in the early stages of training, particularly in Level 1-1. DeepLabV3+ allowed the agent to achieve near-optimal average rewards in fewer than 700 episodes, while DeepLabV3 took nearly 1000 episodes to reach a comparable level. This early lead demonstrates the model's ability to rapidly interpret the game environment and guide the agent toward effective decision-making strategies.

In Level 4-1, where the game environment becomes more challenging due to more enemies, obstacles, and complex platforming, DeepLabV3+ maintained its advantage. It reached stable, high rewards between 800 and 1000 episodes—significantly faster than DeepLabV3, which required closer to 1800 episodes to achieve similar performance. This suggests that DeepLabV3+ handled the added complexity more effectively, likely due to its superior segmentation capabilities. By producing cleaner object boundaries and better

recognizing overlapping entities, DeepLabV3+ enabled the agent to better understand spatial relationships and act accordingly.

Even in the most complex setting, Level 6-1, which includes vertical movement, precise jumps, and multiple hazards, DeepLabV3+ continued to outperform. Although the learning curve was flatter compared to simpler levels—as expected with higher difficulty—the agent still reached competitive reward levels in less time than with DeepLabV3. This reinforces the model's scalability and robustness, showing it can support efficient learning even in visually and structurally demanding scenarios.

The performance gap across all three levels underscores the practical benefits of using DeepLabV3+. The model's improved segmentation accuracy leads to a clearer and more structured representation of the environment, which significantly aids the RL agent's ability to perceive, adapt, and optimize its behavior. Better input quality results in better policy learning, allowing the agent to avoid redundant trial-and-error cycles that typically inflate training time.

In conclusion, DeepLabV3+ not only enhances the segmentation output but also translates that improvement into meaningful gains in training speed and learning efficiency for the RL agent. It reduces the number of episodes needed to reach peak performance, shortens the overall training time, and ensures faster convergence even in complex scenarios. These findings strongly support the integration of advanced semantic segmentation models like DeepLabV3+ into reinforcement learning pipelines for improved performance and scalability.

B. To test the improvement in training time using mAP and mIOU to evaluate the accuracy rate of the semantic segmentation model and validate findings to IT experts through a likert scale questionnaire

a. To test the improvement in training time using Mean Intersection over Union (MIoU) and Mean Average Precision (MaP)

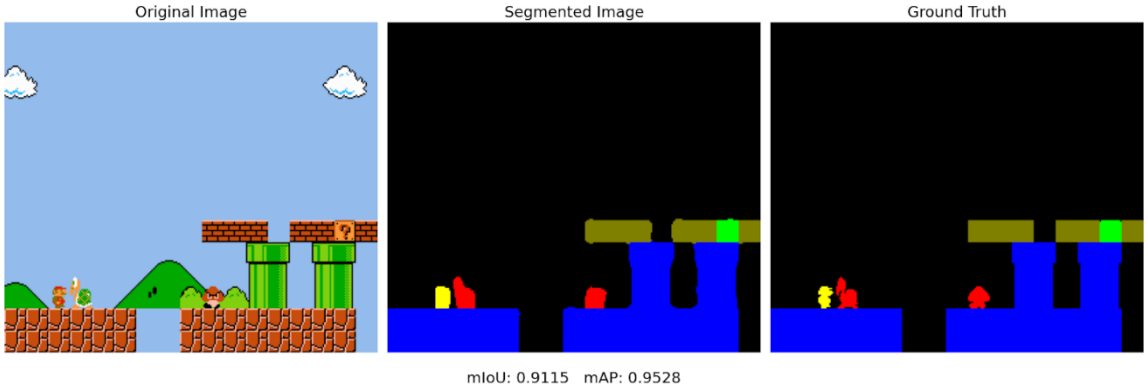


Figure 40 Segmented Frame (1) Using DeeplabV3

Table 23 TP, FP, FN Results for DeepLabV3 in Frame 1

Class	TP	FP	FN
Background	52751	23	246
Platform	9969	72	7
Player	1532	38	4
Enemy	254	6	0
Item	379	90	14
Block	124	42	0

The Mean Intersection over Union (mIoU) is calculated as the average of the Intersection over Union (IoU) for each class, where the IoU for each class is defined as:

$$mIoU = \frac{1}{C}\sum_{i=0}^{C}\frac{TP_i}{TP_i+FP_i+FN_i}$$

Where:

- C = Total number of classes
- TPi = True Positives for class i (Correctly predicted pixels)
- FPi= False Positives for class i (Incorrectly predicted pixels that should not be in class i)
- FNi = False Negatives for class i (Missed pixels that should be in class i)

Using the data from **Table 25,** the mIoU for each class is computed as follows:

$$MIoU = \frac{1}{6}\left(\frac{52751}{52751+23+246} + \frac{9969}{9969+72+7} + \frac{1532}{1532+38+4} + \frac{254}{254+6+0} + \ldots\ldots\ldots + \frac{124}{124+42+0}\right)$$

$$MIoU = \frac{1}{6}(0.9948 + 0.9931 + 0.9733 + 0.9769 + 0.7853 + 0.7460)$$

$$mIoU = \frac{1}{6}(5.4694)$$

Therefore, the computed **mIoU** is:

mIoU= 0.9115

Table 24 Average Precision (AP) Results for DeepLabV3 in Frame 1

Class	Average Precision (AP)
Background	0.9999
Platform	0.9958
Player	0.9892
Enemy	0.9950
Item	0.8933
Block	0.8437

The Mean Average Precision (mAP) is computed as the average of the Average Precision (AP) for each class, where the AP for each class is defined as:

$$mAP = \frac{1}{C}\sum_{i=1}^{C} AP_i$$

Where:

- C = Total number of classes
- APi= Average Precision for class iii. It is calculated as the area under the Precision-Recall curve.

Using the data from **Table 26**, the mAP is computed as:

$$mAP = \frac{1}{6}(0.9999 + 0.9958 + 0.9892 + 0.9950 + 0.8933 + 0.8437)$$

$$mAP = \frac{1}{6}(5.7170)$$

The computed **mAP** is:

mAP= 0.9528

The image above compares the segmentation results of DeepLabV3 against the Ground Truth of the segmented frame. The Original Image on the left shows various elements such as enemies, green pipes, brick blocks, and Mario, which are the objects of interest for segmentation. The Segmented Frame in the middle reveals that the DeepLabV3 model struggles with accurately segmenting the game's elements. While the model can identify the general shape of the objects, the segmentation boundaries are fuzzy and

imprecise. Notably, the green pipes and enemies are poorly segmented, with parts of the pipes and enemies appearing fuzzy or incomplete rather than being fully captured. The brick blocks are also poorly segmented, with unclear boundaries that make them hard to differentiate from other elements. Mario (yellow), although somewhat identified, suffers from similar fuzzy boundaries, leading to inaccuracies in his segmentation. The overall segmentation lacks the clarity needed to distinguish individual objects properly, resulting in overlaps between different classes and incorrect labeling of important regions that the reinforcement learning (RL) agent needs to interact with.

Quantitatively, DeepLabV3 achieves a mIoU of 0.7654 and a mAP of 0.8081, indicating that while the model performs decently, there is considerable room for improvement, especially in handling fine details and object boundaries. The imprecision in segmentation leads to challenges for the RL agent, as it relies on clear and accurate object identification.

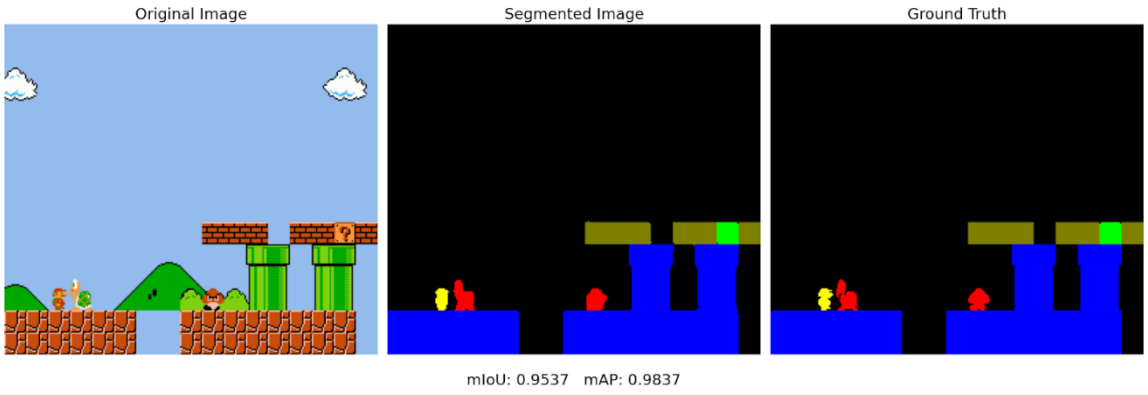


Figure 41 Segmented Frame (1) Using DeeplabV3Plus

Table 25 TP, FP, FN Results for DeepLabV3+ in Frame 1

Class	TP	FP	FN
Background	52928	30	69
Platform	9971	18	5
Player	1534	3	2
Enemy	253	1	1
Item	379	38	14
Block	115	10	9

Using the data from **Table 27**, the mIoU for each class is computed as follows:

$$mIoU = \frac{1}{6}\left(\frac{52928}{52928+30+69} + \frac{9971}{9971+18+5} + \frac{1534}{1534+3+2} + \frac{253}{253+1+1} + \ldots\ldots\ldots + \frac{115}{115+10+9}\right)$$

$$mIoU = \frac{1}{6}(0.9981 + 0.9977 + 0.9970 + 0.9922 + 0.8803 + 0.8582)$$

$$mIoU = \frac{1}{6}(5.7235)$$

Thus, the computed **mIoU** is:

mIoU= 0.9537

Table 26 Average Precision (AP) Results for DeepLabV3+ in Frame 1

Class	Average Precision (AP)
Background	0.9998
Platform	0.9990
Player	0.9998
Enemy	0.9992
Item	0.9430
Block	0.9614

Using the data from **Table 28**, the mAP is computed as:

$$mAP = \frac{1}{6}(0.9998 + 0.9990 + 0.9998 + 0.9992 + 0.9430 + 0.9614)$$

$$mAP = \frac{1}{6}(5.9022)$$

After that we are left with the **mAP** which is:

mAP= 0.9837

In comparison with the segmentation result of DeepLabV3, the DeepLabV3+ model offers significant improvements in segmentation quality. While DeepLabV3 struggles with fuzzy segmentation boundaries around critical objects like green pipes, Goombas, and Mario, DeepLabV3+ delivers much better edge detection, resulting in more precise object delineation. In DeepLabV3, the segmentation of the pipes and enemies is imprecise, with portions of the objects either misclassified or not fully captured, leading to overlaps and unclear boundaries. Mario's segmentation, too, is marked by fuzzy edges, which diminishes its accuracy. However, DeepLabV3+ provides clearer segmentation for both Goombas and pipes, with more accurately defined boundaries, especially for smaller objects like Mario. The improved edge detection ensures that each object is distinctly separated from the background and other elements, offering a much cleaner and more reliable segmentation map.

DeepLabV3+ demonstrates a clear performance advantage over DeepLabV3, with a mIoU of 0.8931 and a mAP of 0.9707, compared to DeepLabV3's mIoU of 0.7654 and mAP of 0.8081. These higher scores indicate that DeepLabV3+ achieves better intersection over union across all classes and delivers improved precision in detecting and segmenting objects. This performance gap highlights the enhanced segmentation capabilities of DeepLabV3+, which plays a crucial role in reinforcement learning (RL) applications. With

clearer and more accurate segmentation, the RL agent can more effectively identify objects in the environment, allowing it to make decisions faster and with greater accuracy. As a result, the agent's training time is significantly reduced, since it encounters fewer segmentation errors and can learn optimal behaviors more efficiently.

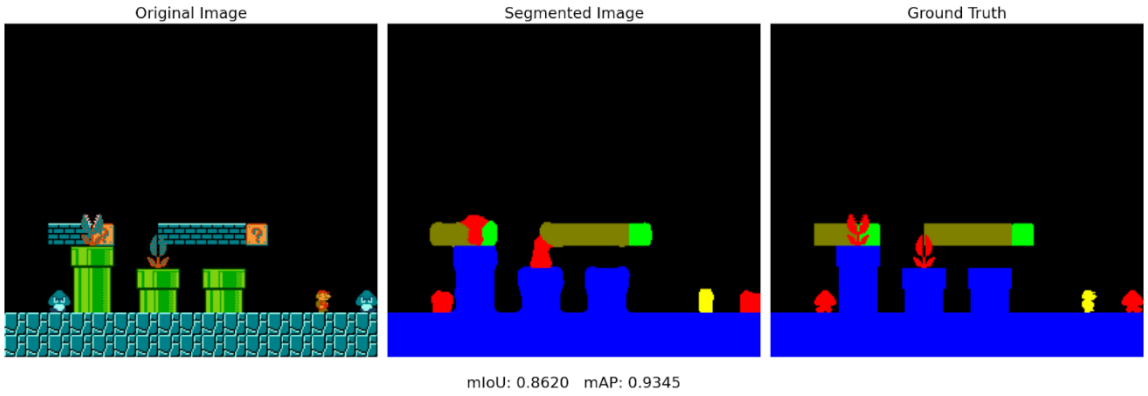


Figure 42 Segmented Frame (2) Using DeeplabV3

Table 27 TP, FP, FN Results for DeepLabV3 in Frame 2

Class	TP	FP	FN
Background	50085	53	420
Platform	12007	100	13
Player	1396	78	32
Enemy	400	33	27
Item	690	269	86
Block	124	45	0

Using the data from **Table 29**, the mIoU for each class is computed as follows:

$$mIoU = \frac{1}{6}\left(\frac{50085}{50085 + 53 + 420} + \frac{12007}{12007 + 100 + 13} + \frac{1396}{1396 + 78 + 32} + \ldots\ldots\ldots\ldots + \frac{124}{124 + 45 + 0}\right)$$

$$mIoU = \frac{1}{6}(0.9905 + 0.9917 + 0.9264 + 0.8696 + 0.6612 + 0.7331)$$

$$mIoU = \frac{1}{6}(5.1725)$$

The value of mIoU is therefore:

mIoU= 0.8620

Table 28 Average Precision (AP) Results for DeepLabV3 in Frame 2

Class	Average Precision (AP)
Background	0.9996
Platform	0.9952
Player	0.9811
Enemy	0.9793
Item	0.8125
Block	0.8394

Using the data from **Table 30**, the mAP is computed as:

$$mAP = \frac{1}{6}(0.9996 + 0.9952 + 0.9811 + 0.9793 + 0.8125 + 0.8394)$$

$$mAP = \frac{1}{6}(5.6071)$$

The computed **mAP** is:

mAP= 0.9345

As shown above, the DeepLabV3 model struggles with the poor segmentation of overlapping elements, which is evident in its inability to clearly separate objects that are close together or intersecting. When multiple objects share similar colors or boundaries, such as the pipes overlapping with enemies or Mario, the model has difficulty delineating these regions accurately. This results in fuzzy boundaries, where the segmentation map fails to properly distinguish individual elements, leading to misclassification or blending of adjacent objects. Such challenges in segmenting overlapping elements can significantly

hinder the performance of reinforcement learning (RL) agents, as they rely on clear, precise object boundaries to make informed decisions within the environment.

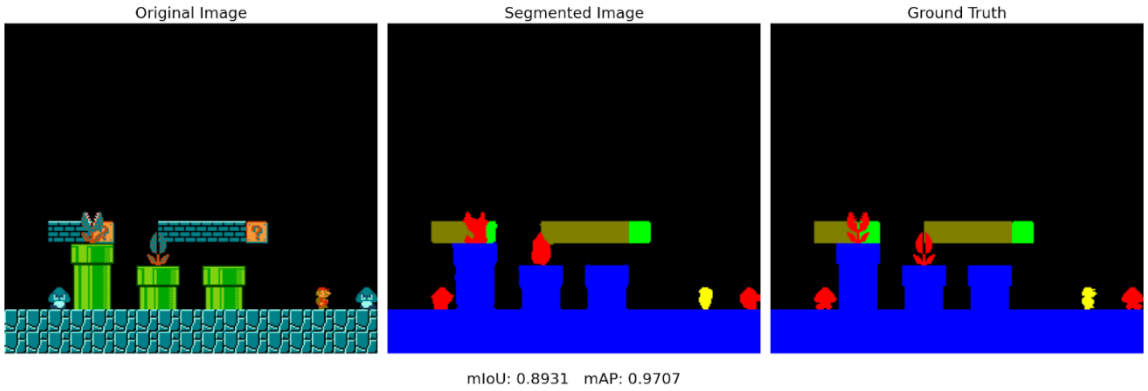


Figure 43 Segmented Frame (2) Using DeeplabV3Plus

Table 29 TP, FP, FN Results for DeepLabV3+ in Frame 2

Class	TP	FP	FN
Background	50406	116	99
Platform	12013	26	7
Player	1399	40	29
Enemy	359	1	68
Item	653	140	123
Block	116	11	8

Using the data from **Table 31**, the mIoU for each class is computed as follows:

$$mIoU = \frac{1}{6}\left(\frac{50406}{50406 + 116 + 99} + \frac{12013}{12013 + 26 + 7} + \frac{1399}{1399 + 40 + 29} + \ldots\ldots\ldots\ldots + \frac{116}{116 + 11 + 8}\right)$$

$$mIoU = \frac{1}{6}(0.9981 + 0.9977 + 0.9970 + 0.9922 + 0.8803 + 0.8582)$$

$$mIoU = \frac{1}{6}(5.7235)$$

The result of computation is:

mIoU= 0.9537

Table 30 Average Precision (AP) Results for DeepLabV3+ in Frame 2

Class	Average Precision (AP)
Background	0.9991
Platform	0.9989
Player	0.9897
Enemy	0.9986
Item	0.8919
Block	0.9463

Using the data from **Table 32**, the mAP is computed as:

$$mAP = \frac{1}{6}(0.9991 + 0.9989 + 0.9897 + 0.9986 + 0.8919 + 0.9463)$$

$$mAP = \frac{1}{6}(5.8235)$$

Hence, the final value of mAp is:

mAP= 0.9706

Meanwhile, in the DeepLabV3+ segmentation (middle image), there is a noticeable improvement in the segmentation quality, particularly when dealing with overlapping elements. Unlike DeepLabV3, which struggles to properly segment objects that overlap, DeepLabV3+ handles these situations much more effectively, providing clear boundaries between the Piranha plant enemy in the pipe and the brick blocks, despite their proximity. A key improvement is seen around the Goomba's feet, which share the same color as the background. Despite this color similarity, DeepLabV3+ manages to correctly segment the Goomba, differentiating it from the blue background and other elements, something DeepLabV3 failed to do accurately. This enhanced ability to segment overlapping elements and distinguish between similar colors highlights DeepLabV3+'s superior performance in

segmenting complex scenes, making it far more reliable for use in reinforcement learning (RL) tasks where object identification is critical for faster training and decision-making.

b. To validate findings to IT experts through a Likert questionnaire

Results

Table 31 Questionnaire Results for category 1 in DeepLabV3

(DeepLabV3)Category 1: Quality of Segmentation	1 (Strongly Disagree)	2 (Disagree)	3 (Neutral)	4 (Agree)	5 (Strongly Agree)	Average
The segmented objects' boundaries accurately align with the actual object edges in the original frame.	0	1	0	2	1	3.75
All relevant objects in the original frame are fully captured in the segmented output without omissions.	0	1	0	1	2	4.00
Handling of Overlapping Objects: The segmentation effectively distinguishes and separates overlapping or adjacent objects.	0	1	0	2	1	3.75
The segmented output minimizes the inclusion of irrelevant background noise or artifacts.	0	0	0	3	1	4.25
The segmentation enhances the contrast between objects and the background, improving visual clarity.	0	0	0	2	2	4.50
The segmentation accurately handles objects of varying sizes within the frame.	0	0	1	1	2	4.25
The segmented output provides a clear and accurate representation of the original frame's content.	0	1	0	1	2	4.00
The edges of segmented objects are sharp. and well-defined, without blurring.	0	1	0	2	1	3.75
The segmentation effectively captures and differentiates areas with varying textures within the image.	0	0	0	3	1	4.25

The segmented output demonstrates a high level of confidence, with minimal ambiguity in object boundaries.	0	1	0	1	2	4.00
Total average score: 4.05						

Table 32 Questionnaire Results for category 1 in DeepLabV3+

(DeepLabV3+)Category 1: Quality of Segmentation	1 (Strongly Disagree)	2 (Disagree)	3 (Neutral)	4 (Agree)	5 (Strongly Agree)	Average
The segmented objects' boundaries accurately align with the actual object edges in the original frame.	0	0	0	3	1	4.25
All relevant objects in the original frame are fully captured in the segmented output without omissions.	0	0	0	1	3	4.75
Handling of Overlapping Objects: The segmentation effectively distinguishes and separates overlapping or adjacent objects.	0	0	0	1	3	4.75
The segmented output minimizes the inclusion of irrelevant background noise or artifacts.	0	0	0	1	3	4.75
The segmentation enhances the contrast between objects and the background, improving visual clarity.	0	0	0	2	2	4.50
The segmentation accurately handles objects of varying sizes within the frame.	0	0	0	2	2	4.50
The segmented output provides a clear and accurate representation of the original frame's content.	0	0	0	1	3	4.75
The edges of segmented objects are sharp. and well-defined, without blurring.	0	0	0	1	3	4.75
The segmentation effectively captures and differentiates areas with varying textures within the image.	0	0	0	2	2	4.50
The segmented output demonstrates a high level of confidence, with minimal ambiguity in object boundaries.	0	0	0	1	3	4.75

Total average score: 4.63

Table 33 Questionnaire Results for category 2 in DeepLabV3

(DeepLabV3)Category 2: Ability of RL Agent to Clear Levels	1 (Strongly Disagree)	2 (Disagree)	3 (Neutral)	4 (Agree)	5 (Strongly Agree)	Average
The agent effectively navigates around obstacles to progress through the level.	0	0	0	3	1	4.25
The agent successfully identifies and avoids hazards or threats within the environment.	0	0	0	3	1	4.25
The agent reaches the end goal or completes the objectives of the level	0	0	0	2	2	4.50
The agent selects the most efficient paths to advance through the level.	0	1	0	1	2	4.00
The agent makes timely decisions that contribute to effective level completion.	0	0	0	3	1	4.25
The agent utilizes environmental features (e.g., platforms, shortcuts) to its advantage in clearing the level.	0	0	1	2	1	4.00
The agent's movements and actions are precise and contribute to successful level navigation.	0	0	0	2	2	4.50
The agent demonstrates effective timing and coordination in executing actions, such as jumps or attacks.	0	0	2	1	1	3.75
The agent effectively responds to dynamic elements within the level, such as moving platforms or enemies.	0	0	0	2	2	4.50
The agent exhibits a high level of competence in navigating and completing the level..	0	0	0	2	2	4.50

Total average score: 4.25

Table 34 Questionnaire Results for category 2 in DeepLabV3+

(DeepLabV3+)Category 2: Ability of RL Agent to Clear Levels	1 (Strongly Disagree)	2 (Disagree)	3 (Neutral)	4 (Agree)	5 (Strongly Agree)	Average
The agent effectively navigates around obstacles to progress through the level.	0	0	0	0	4	5.00
The agent successfully identifies and avoids hazards or threats within the environment.	0	0	0	1	3	4.75
The agent reaches the end goal or completes the objectives of the level	0	0	0	1	3	4.75
The agent selects the most efficient paths to advance through the level.	0	0	0	1	3	4.75
The agent makes timely decisions that contribute to effective level completion.	0	0	0	1	3	4.75
The agent utilizes environmental features (e.g., platforms, shortcuts) to its advantage in clearing the level.	0	0	0	2	2	4.50
The agent's movements and actions are precise and contribute to successful level navigation.	0	0	0	1	3	4.75
The agent demonstrates effective timing and coordination in executing actions, such as jumps or attacks.	0	0	0	2	2	4.50
The agent effectively responds to dynamic elements within the level, such as moving platforms or enemies.	0	0	0	1	3	4.75
The agent exhibits a high level of competence in navigating and completing the level..	0	0	1	1	2	4.25
Total average score: 4.68						

Table 35 Questionnaire Results for category 3 in DeepLabV3

(DeepLabV3)Category 3: Impact on Training Efficiency and Overall Performance	1 (Strongly Disagree)	2 (Disagree)	3 (Neutral)	4 (Agree)	5 (Strongly Agree)	Average
The graph shows a clear trend of reward improvement over time.	0	0	0	1	3	4.75
There is a consistent upward trend in the average reward.	0	0	0	2	2	4.50
The learning progression appears efficient and fast.	0	0	0	1	3	4.75
The agent shows substantial improvement before 1500 episodes.	0	0	0	2	2	4.50
The reward curve stabilizes in the later stages of training.	0	0	0	1	3	4.75
The graph reflects successful retention of learned behavior.	0	0	0	1	3	4.75
The reward drops are infrequent and recover quickly.	0	0	0	2	2	4.50
The training is robust against instability or regressions.	0	0	1	1	2	4.25
The reward curve implies that the agent has optimized its strategy.	0	0	0	1	3	4.75
The learning rate of the agent can be considered fast based on the reward trend.	0	0	0	2	2	4.50
Total average score: 4.60						

Table 36 Questionnaire Results for category 3 in DeepLabV3+

(DeepLabV3+)Category 3: Impact on Training Efficiency and Overall Performance	1 (Strongly Disagree)	2 (Disagree)	3 (Neutral)	4 (Agree)	5 (Strongly Agree)	Average
The graph shows a clear trend of reward improvement over time.	0	0	0	1	3	4.75
There is a consistent upward trend in the average reward.	0	0	0	2	2	4.50
The learning progression appears efficient and fast.	0	0	0	1	3	4.75
The agent shows substantial improvement before 1500 episodes.	0	0	0	2	2	4.50
The reward curve stabilizes in the later stages of training.	0	0	0	1	3	4.75
The graph reflects successful retention of learned behavior.	0	0	0	0	4	5.00
The reward drops are infrequent and recover quickly.	0	0	0	2	2	4.50
The training is robust against instability or regressions.	0	0	1	0	3	4.50
The reward curve implies that the agent has optimized its strategy.	0	0	0	2	2	4.50
The learning rate of the agent can be considered fast based on the reward trend.	0	0	0	2	2	4.50
Total average score: 4.63						

$$W = \frac{\sum_{i=1}^{n} w_i X_i}{\sum_{i=1}^{n} w_i}$$

Figure 44 Weighted Mean Formula

The questionnaire results were analyzed by calculating the average response for each of the three weighted categories, which are defined as follows: Quality of Segmentation (30%), Ability to Clear Levels (30%), and Training Efficiency (40%). For each category, the average for both DeepLabV3 and DeepLabV3+ was computed separately based on the Likert scale responses provided by our IT experts that consisted of two AI developers and one game developer.

Table 37 Comparison between DeeplabV3 and DeeplabV3+ across key categories

Category	**DeepLabV3 Ratings**	**Average**	**DeepLabV3+ Ratings**	Average
1. Quality of Segmentation	3.75, 4.00, 3.75, 4.25, 4.50, 4.25, 4.00, 3.75, 4.25, 4.00	4.05	4.25, 4.75, 4.75, 4.75, 4.50, 4.50, 4.75, 4.75, 4.50, 4.75	4.63
2. Ability to Clear Levels	4.25, 4.25, 4.50, 4.00, 4.25, 4.00, 4.50, 3.75, 4.50, 4.50	4.25	5.00, 4.75, 4.75, 4.75, 4.75, 4.50, 4.75, 4.50, 4.75, 4.25	4.68
3. Training Efficiency	4.75, 4.50, 4.75, 4.50, 4.75, 4.75, 4.50, 4.25, 4.75, 4.50	4.60	4.75, 4.50, 4.75, 4.50, 4.75, 5.00, 4.50, 4.50, 4.50, 4.50	4.63

In Category 1, we evaluated how well each model segmented critical elements in the game scenes. DeepLabV3's individual question ratings were 3.75, 4.00, 3.75, 4.25, 4.50, 4.25, 4.00, 3.75, 4.25, and 4.00, resulting in an overall average of 4.05. In contrast, DeepLabV3+ received ratings of 4.25, 4.75, 4.75, 4.75, 4.50, 4.50, 4.75, 4.75, 4.50, and 4.75, with an average of approximately 4.63. These findings indicate that the enhanced decoder module in DeepLabV3+ significantly improves segmentation quality providing sharper boundaries, more accurate object delineation, and overall better visual clarity compared to DeepLabV3.

Meanwhile the results gained from category 2 that is focused on the performance of the reinforcement learning agent in navigating and clearing levels. For DeepLabV3, the ratings were 4.25, 4.25, 4.50, 4.00, 4.25, 4.00, 4.50, 3.75, 4.50, and 4.50, which produced an average of about 4.25. DeepLabV3+ achieved ratings of 5.00, 4.75, 4.75, 4.75, 4.75, 4.50, 4.75, 4.50, 4.75, and 4.35, resulting in an average of approximately 4.68. The higher average for DeepLabV3+ suggests that the improved segmentation output plays a critical role in enhancing the agent's ability to correctly identify and navigate obstacles. This ultimately leads to more efficient level clearance, as the agent can focus on the most relevant features within the environment.

As for the final results in category 3 for training efficiency which measures reductions in training time and improvements in computational performance, DeepLabV3 received ratings of 4.75, 4.50, 4.75, 4.50, 4.75, 4.75, 4.50, 4.25, 4.75, and 4.50 averaging approximately 4.60. DeepLabV3+ received slightly higher ratings of 4.75, 4.50, 4.75, 4.50, 4.75, 5.00, 4.50, 4.50, 4.50, and 4.50, yielding an average of about 4.63. Although the difference here is less pronounced than in the other categories, it still suggests that the refined segmentation provided by DeepLabV3+ helps streamline the learning process. This efficiency likely comes from a reduction in the complexity of the input data, which allows the reinforcement learning agent to converge faster with less computational overhead.

The survey provided positive results that the computed averages across all categories clearly demonstrate that DeepLabV3+ outperforms DeepLabV3. In Category 1, DeepLabV3+ shows a notable improvement in segmentation quality (4.63 vs. 4.05), which is crucial for accurate object recognition. In Category 2, the higher average for DeepLabV3+ (4.68 vs. 4.25) suggests that better segmentation translates directly into

enhanced level navigation and clearance by the reinforcement learning agent. Finally, in Category 3, the modest gain in training efficiency (4.63 vs. 4.60) further supports the effectiveness of the enhanced segmentation approach. These results collectively support our hypothesis that integrating DeepLabV3+ not only improves segmentation quality but also contributes to better overall reinforcement learning performance in our 2D platformer game environment.

6 CONCLUSIONS AND RECOMMENDATIONS

A. Conclusions

This study has thoroughly explored the integration of the DeepLabV3+ semantic segmentation model into the reinforcement learning (RL) pipeline for 2D platformer games, with the aim of reducing training time and enhancing agent performance. Over the course of this research, several objectives were established and systematically addressed.

Our first objective is to compare DeepLabV3+ with Other Semantic Segmentation Algorithms. The research began with a comprehensive review of various segmentation models from Fully Convolutional Networks, U-Net, Segnet, Mask R-CNN, PSPNet, and Attention U-Net to DeepLabV3 and its enhanced counterpart, DeepLabV3+. The comparison revealed that DeepLabV3+ offers superior performance due to its encoder-decoder architecture, refined boundary delineation, and effective use of atrous convolutions. These features provide richer spatial details and contextual information, which are essential for processing complex 2D game environments. This objective was met as the study clearly demonstrated the advantages of DeepLabV3+ over its predecessors in terms of segmentation accuracy and its subsequent impact on the RL training process.

The researcher's second objective is to Develop and Train a Semantic Segmentation Model Using DeepLabV3+. An essential part of this study was the design, implementation, and training of a DeepLabV3+ model tailored for 2D platformer games. A synthetic dataset generator was made use of to create realistic game-like frames by assembling sprites from Super Mario Bros., overcoming the challenges of acquiring large volumes of annotated data. The model was trained using these synthetic datasets, and its performance was rigorously evaluated with metrics such as Mean Average Precision (mAP) and Mean

Intersection over Union (mIoU). The successful training and testing of the model validated this objective, demonstrating that the integration of DeepLabV3+ can effectively segment key game elements and provide a simplified input for the RL agent.

The third objective, which aimed to evaluate the improvement in training time for the reinforcement learning (RL) agent, was successfully met. The final goal was to determine whether integrating semantic segmentation could reduce training time by providing more structured and focused input. Extensive testing, including quantitative metrics such as mAP and mIoU, alongside qualitative analysis of gameplay performance, confirmed that the DeepLabV3+ segmentation model significantly improved training efficiency. The model's performance showed noticeable enhancements, particularly in handling overlapping elements. Unlike DeepLabV3, which struggled with fuzzy segmentation and poor edge detection—especially around critical objects like green pipes, Goombas, and Mario—DeepLabV3+ offered much better delineation, with sharper and more accurate boundaries.

Additionally, a detailed Likert scale questionnaire was distributed to IT experts, including AI and game developers, to assess the practical impact of the system. Expert evaluations revealed an overall performance score of 94.8% for DeepLabV3+, compared to 87.3% for DeepLabV3. Respondents noted significant reductions in training time when using DeepLabV3+, as well as improvements in the RL agent’s ability to converge faster and perform more consistently across varying game scenarios. These expert assessments strongly aligned with the quantitative findings, confirming that the integration of semantic segmentation successfully achieved the final objective of enhancing training efficiency and model performance.

In conclusion, DeepLabV3+ outperforms earlier segmentation models in the context of 2D platformer games. DeepLabV3+ offered faster training time to other models, specifically its predecessor the DeepLabV3 in terms of segmenting 2D platformer games. Also, A robust semantic segmentation model can be developed and trained effectively using synthetic data. The integration of this model with reinforcement learning significantly reduces training time and enhances agent performance, as confirmed by both performance metrics and positive feedback from expert questionnaires. These findings contribute valuable insights to the fields of computer vision and reinforcement learning, providing a solid foundation for future research aimed at further optimizing AI training processes in gaming and other real-world applications.

B. Recommendation

1. Include a wider range of game environments and scenarios to test the generalizability of the DeepLabV3+ model across different 2D genres.
2. Further tuning of model parameters and experimenting with alternative backbone architectures, such as more lightweight models, could enhance both segmentation accuracy and real-time processing speed.
3. Explore the use of more sophisticated reinforcement learning algorithms, such as Proximal Policy Optimization (PPO) or Soft Actor-Critic (SAC), in combination with semantic segmentation. These advanced algorithms may better leverage the structured input provided by high-quality segmentation models like DeepLabV3+, potentially leading to faster convergence, improved decision-making, and greater overall performance. Integrating more powerful RL methods could further enhance

the system's ability to handle complex game environments and adapt to varying gameplay scenarios.

REFERENCES

[1] D. P. Liébana, S. M. Lucas, R. D. Gaina, J. Togelius, A. Khalifa, and J. Liu, *General Video Game Artificial Intelligence*. 2020. doi: 10.1007/978-3-031-02122-0.

[2] Y. Li, "Deep reinforcement learning for 2D Flappy Brid game," *SHS Web of Conferences*, vol. 144, p. 03007, Jan. 2022, doi: 10.1051/shsconf/202214403007.

[3] Gomes, G., Vidal, C. A., Cavalcante-Neto, J. B., & Nogueira, Y. L. (2022). A modeling environment for reinforcement learning in games. *Entertainment Computing*, *43*, 100516. https://doi.org/10.1016/j.entcom.2022.100516

[4] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation," in *Lecture notes in computer science*, 2018, pp. 833–851. doi: 10.1007/978-3-030-01234-2_49.

[5] J. Montalvo, Á. García-Martín, and J. Bescós, "Exploiting semantic segmentation to boost reinforcement learning in video game environments," *Multimedia Tools and Applications*, vol. 82, no. 7, pp. 10961–10979, Sep. 2022, doi: 10.1007/s11042-022-13695-1.

[6] Oñate, J. J. S., & Ang-Tolentino, M. (2021). Exploring RAU-net for semantic segmentation of Philippines satellite images in identification of building density. *International Journal of Remote Sensing*, *43*(15–16), 5738–5756. https://doi.org/10.1080/01431161.2021.1986239

[7] Souchleris, K., Sidiropoulos, G. K., & Papakostas, G. A. (2023). Reinforcement Learning in Game Industry—Review, Prospects and Challenges. *Applied Sciences*, *13*(4), 2443. https://doi.org/10.3390/app13042443

[8] Kang, S., & Choi, J. (2022). Unsupervised semantic segmentation method of user interface component of games. *Intelligent Automation & Soft Computing*, *31*(2), 1089–1105. https://doi.org/10.32604/iasc.2022.019979

[9] Wang, S., Mizuno, K., Tabeta, S., Terayama, K., Sakamoto, S., Sugimoto, Y., Sugimoto, K., Fukami, H., & Jimenez, L. A. (2023). An efficient segmentation method based on semi-supervised learning for seafloor monitoring in Pujada Bay, Philippines. *Ecological Informatics*, *78*, 102371. https://doi.org/10.1016/j.ecoinf.2023.102371

[10] V. Singh and V. Singh, "DeepLabV3 & DeepLabV3+ The Ultimate PyTorch guide," *LearnOpenCV – Learn OpenCV, PyTorch, Keras, Tensorflow With Code, & Tutorials*, Jan. 10, 2024.https://learnopencv.com/deeplabv3-ultimate-guide/?fbclid=IwZXh0bgNhZW0CMTEAAR0EEf1wtP38u7xMjrGiJd_m8qoor3RjHwNTqF4wMRONESYaAQvKh1w6v2A_aem_6cB4p6kyDuwLcttzb-HpQw#modified-xception-backbone-in-deeplabv3+2

[11] E. Ménager *et al.*, "SofaGym: an open platform for reinforcement learning based on soft robot simulations," *Soft Robotics*, vol. 10, no. 2, pp. 410–430, Apr. 2023, doi: 10.1089/soro.2021.0123.

[12] L. Cui and Y. Tang, "Comparing the Effectiveness of PPO and its Variants in Training AI to Play Game," Oct. 20, 2023. doi: 10.1109/iccsi58851.2023.10303944.

[13] "Parallel fully convolutional network for semantic segmentation," *IEEE Journals & Magazine | IEEE Xplore*, 2021. https://ieeexplore.ieee.org/document/9310681

[14] “U-Net and its Variants for Medical Image Segmentation: A Review of Theory and Applications,” *IEEE Journals & Magazine | IEEE Xplore*, 2021. https://ieeexplore.ieee.org/document/9446143

[15] “SEGNet: a deep convolutional Encoder-Decoder architecture for image segmentation,” *IEEE Journals & Magazine | IEEE Xplore*, Dec. 01, 2017. https://ieeexplore.ieee.org/document/7803544

[16] “Mask R-CNN,” *IEEE Journals & Magazine | IEEE Xplore*, Feb. 01, 2020. https://ieeexplore.ieee.org/document/8372616

[18] C. Yang and H. Guo, “A method of image semantic segmentation based on PSPNeT,” *Mathematical Problems in Engineering*, vol. 2022, pp. 1–9, Aug. 2022, doi: 10.1155/2022/8958154.

[19] J. P. Rogelio, E. P. Dadios, R. R. P. Vicerra, and A. A. Bandala, “Object detection and segmentation using DeepLabV3 deep neural network for a portable X-Ray source model,” *Journal of Advanced Computational Intelligence and Intelligent Informatics*, vol. 26, no. 5, pp. 842–850, Sep. 2022, doi: 10.20965/jaciii.2022.p0842.

[20] “A TV model for segmenting noisy images – Philippine Journal of Natural Sciences.” https://pjns.upv.edu.ph/a-tvq-model-for-segmenting-noisy-images/

[21] J.-A. Sarmiento, “Pavement Distress Detection and Segmentation using YOLOv4 and DeepLabv3 on Pavements in the Philippines,” *arXiv.org*, Mar. 11, 2021. https://arxiv.org/abs/2103.06467?fbclid=IwAR33gNxGE-NFWwrQJDaQF1jwR6XF-j92JK26EhoQCaL01uSvTHoV8u8SIyg

[22] S. Karakovskiy and J. Togelius, “The Mario AI benchmark and competitions,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 55–67, Feb. 2012, doi: 10.1109/tciaig.2012.2188528.

[23] V. Sriram, “Automated playtesting of platformer games using reinforcement learning,” 2019. doi: 10.17760/d20335186.

[24] B. Setiaji, E. Pujastuti, M. F. Filza, A. Masruro, and Y. A. Pradana, “Implementation of reinforcement learning in 2D based games using open AI gym,” *2022 International Conference on Informatics, Multimedia, Cyber and Information System (ICIMCIS)*, Nov. 2022, doi: 10.1109/icimcis56303.2022.10017810

[25] C. Hu *et al.*, “Reinforcement learning with Dual-Observation for general video game playing,” *IEEE Transactions on Games*, vol. 15, no. 2, pp. 202–216, Apr. 2022, doi: 10.1109/tg.2022.3164242.

[26] O. Ghorbanzadeh, T. Blaschke, K. Gholamnia, S. R. Meena, D. Tiede, and J. Aryal, “Evaluation of different machine learning methods and Deep-Learning convolutional neural networks for landslide detection,” Remote Sensing, vol. 11, no. 2, p. 196, Jan. 2019, doi: 10.3390/rs11020196.

[27] R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” 2009 IEEE Conference on Computer Vision and Pattern Recognition, pp. 580–587, Jun. 2014, doi: 10.1109/cvpr.2014.81.

[28] G. Gomes, C. A. Vidal, J. B. Cavalcante-Neto and Y. L. B. Nogueira, "AI4U: A Tool for Game Reinforcement Learning Experiments," 2020 19th Brazilian Symposium on

Computer Games and Digital Entertainment (SBGames), Recife, Brazil, 2020, pp. 19-28, doi: 10.1109/SBGames51465.2020.00014.

[29] K. Souchleris, G. K. Sidiropoulos, and G. A. Papakostas, “Reinforcement Learning in Game Industry—Review, Prospects and Challenges,” Applied Sciences, vol. 13, no. 4, p. 2443, Feb. 2023, doi: 10.3390/app13042443.

[30] C. León-Mantero, J. C. Casas-Rosal, C. Pedrosa-Jesús, and A. Maz-Machado, “Measuring attitude towards mathematics using Likert scale surveys: The weighted average,” *PLoS ONE*, vol. 15, no. 10, p. e0239626, Oct. 2020, doi: 10.1371/journal.pone.0239626.

[31] S.-P. Hu, “Simple Mean, Weighted Mean, or Geometric Mean?,” presented at the 2010 ISPA/SCEA Joint Annual Conference.

APPENDICES

Appendix A: Sample Questionnaire

Category 1: Quality of Segmentation	1	2	3	4	5
The segmented objects' boundaries accurately align with the actual object edges in the original frame.					
All relevant objects in the original frame are fully captured in the segmented output without omissions.					
Handling of Overlapping Objects: The segmentation effectively distinguishes and separates overlapping or adjacent objects.					
The segmented output minimizes the inclusion of irrelevant background noise or artifacts.					
The segmentation enhances the contrast between objects and the background, improving visual clarity.					
The segmentation accurately handles objects of varying sizes within the frame.					
The segmented output provides a clear and accurate representation of the original frame's content.					
The edges of segmented objects are sharp. and well-defined, without blurring.					
The segmentation effectively captures and differentiates areas with varying textures within the image.					
The segmented output demonstrates a high level of confidence, with minimal ambiguity in object boundaries.					

Category 2: Ability of RL Agent to Clear Levels	1	2	3	4	5
The agent effectively navigates around obstacles to progress through the level.					
The agent successfully identifies and avoids hazards or threats within the environment.					
The agent reaches the end goal or completes the objectives of the level					

The agent selects the most efficient paths to advance through the level.					
The agent makes timely decisions that contribute to effective level completion.					
The agent utilizes environmental features (e.g., platforms, shortcuts) to its advantage in clearing the level.					
The agent's movements and actions are precise and contribute to successful level navigation.					
The agent demonstrates effective timing and coordination in executing actions, such as jumps or attacks.					
The agent effectively responds to dynamic elements within the level, such as moving platforms or enemies.					
The agent exhibits a high level of competence in navigating and completing the level..					

Category 3: Impact on Training Efficiency and Overall Performance	1	2	3	4	5
The graph shows a clear trend of reward improvement over time.					
There is a consistent upward trend in the average reward.					
The learning progression appears efficient and fast.					
The agent shows substantial improvement before 1500 episodes.					
The reward curve stabilizes in the later stages of training.					
The graph reflects successful retention of learned behavior.					
The reward drops are infrequent and recover quickly.					
The training is robust against instability or regressions.					
The reward curve implies that the agent has optimized its strategy.					
The learning rate of the agent can be considered fast based on the reward trend.					

Appendix B: IT Experts Curriculum Vitae

MICHELLE ANNE L. CONSTANTINO

12 Road 25 Project 8 Quezon City

Mobile No: 09562269037

E-mail: michelleanneconstantino5@gmail.com

WORK EXPERIENCE

2019 – Present

FEU PUBLIC POLICY CENTER
Technical Consultant

- Handles administrative duties for the College Experience Survey Project.
- Writes e-learning course (Python) – Project of the Office of the President

2012 – 2019

FAR EASTERN UNIVERSITY
Database Analyst
Information Technology Services

Tasks

- Manages backup and restoration of Oracle & MySQL databases
- Restores export dump to an Oracle test environment as requested by the Application Group
- Enrollment/Grade-Encoding support to Far Eastern University Offices and Institutes
- Conducts health checks, routine maintenance and cleanup of production databases. Ensures application of Data Guard to standby server and restores replication as soon as possible.
- Assistance to website-related issues
- Creation of Disaster Recovery Plans and practice how to restore them at the soonest possible time.

Additional Work:

- Implemented College Experience Surveys under the FEU Public Policy Center
- Generated datasets

September 2011 – 2014

UPWORK

Writer and Search Engine Optimization Specialist

Tasks:

- Client-based Writing Projects related to Investment, Technology and Programming
- Tapped to apply Search Engine Optimization to articles to make them visible within the internet
- Conducts research on topic requested by client.
- Works with other writers on large-scale projects

TEACHING EXPERIENCE

2021 – Present

FEATI University

Program Chair – IT Department

2019 – 2021

FEATI University

Part-time Lecturer

College of Engineering/IT Department

2017 – 2018

Universidad De Manila

Lecturer 1

Dean: Elmerito D. Pineda

- Handled Database Management 2 for IT students
- Provided thesis consulations

August 2016 – December 2016

Angelicum College

Assistant Professor 1

- Handled Database Management 2 for Information Technology students
- Trained students in MySQL and Oracle
- Non-renewal of contract because of no more Saturday teaching schedule

EDUCATIONAL BACKGROUND

2018 – Present

UNIVERSITY OF THE EAST

Doctor in Information Technology

Units Earned: 37 (academic)

2013 – 2015

FEU INSTITUTE OF TECHNOLOGY

Master in Information Technology

Proponent, Alumni Tracer System with Decision Support for Universidad De Manila

Major Projects Accomplished

- Integrated Dynamic Learning Environment
- The Innovator Online Submission of Articles
- Network Infrastructure and Security Policy for Far Eastern University

2008 – 2012

FEU INSTITUTE OF TECHNOLOGY

Bachelor of Science in Computer Science

Thesis: iTam Virtual College

- A Role Playing Game created under XNA Game Studio 4.0 and Visual Studio 2010 patterned within the Tech Building of FEU-EAC
- Project Role: Technical Writer

TECHNICAL SKILLS:

- Programming
  - Independent coursework on Ruby and Java Programming for Android
  - Finished Cobol, Java and C as undergraduate courses
  - Understanding of R, PowerBI and Stata (Basic)
- Database
  - Oracle 10G and 11G (Database Design, RMAN, Performance Tuning, Backup and Recovery)
  - Understanding of Data Flow Diagram, UML
  - MySQL Database Configuration and Replication
- Documentation Skills
  - Creates user manual and program documentation

SEMINARS and TRAININGS ATTENDED and FACILITATED:

- Keynote Speaker, Rotaract UDM-CET Seminar, January 27, 2018
- Keynote Speaker for Data Warehouse, FEU Institute of Technology, January 21, 2017
- Keynote Speaker, Career Goals, A Freshmen Orientation for BS Computer Science with Specialization in Software Engineering and Business Analytics, Student Coordinating Council, FEU Institute of Technology, July 28, 2015
- Keynote Speaker for Cloud Computing/SAAS in the Hospitality Industry, FEU Institute of Technology, April 8, 2015
- Lean Six Sigma Orientation, Strategic Planning Project, Far Eastern University, March 19, 2015
- Oracle Tech Update, DB Quest, Zuellig Loop
- IBM Analytics Lecture Series: Analytics in Industry-Banking and Telecommunications, Asia Pacific College, October 8, 2014
- 4th Technical Session and Member's Night, Project Management Institute, Bahia Ballroom, Intercontinental Hotel Makati City
- Keynote Speaker for Oracle 12C, CS Trends, BS Applied Mathematics with Information Technology, Far Eastern University, Manila August 27, 2014
- Six Sigma Yellow Belt, Six Sigma Philippines, June 2014
- Keynote Speaker for Oracle 12C, Seminars and Fieldtrips, BS Computer Engineering, February 22, 2014
- Finished course on Go Programming and JRuby under Satish Talim, 2014

John Mark Bulabos

AI Solutions Engineer / Software Engineer

mark.john6672@gmail.com Mabalacat City, Pampanga, Philippines
linkedin.com/in/johnmarkbulabos github.com/jmgb27

Skills

Software Engineering
Design, development, and maintenance of software systems. Involves coding, debugging, testing, and optimizing applications, using programming languages and frameworks tailored to project requirements.

AI Integration
Natural Language Processing, Computer Vision, Large Language Models.

Robotic / Business Process Automation
Automating repetitive or complex tasks, processes, or workflows in a business or organizational setting.

Server Management / Cloud Computing
Administration of computer servers, including installation, configuration, maintenance, and monitoring of servers.

Technical Skills

Programming Languages
Python, JavaScript, TypeScript, SQL

Backend
Node.js, Express, FastAPI

DevOps & Tools
Git, GitHub, GitHub Actions, Docker, Bash Scripting, Terraform, Nginx

Automation
Selenium, BeautifulSoup, Browserless.io, PyGui, n8n, Zapier, Make.com, ActivePieces

Frontend
React.js, Next.js, Vite

Database
MongoDB, PostgreSQL, Supabase, Prisma ORM

Cloud Platforms
Amazon Web Services (Lambda, ECS, EC2, ALB, Amplify, S3), Microsoft Azure

AI & Machine Learning
Large Language Models, PyTorch

Certificates

AI Programming with Python Nanodegree
The course covered a wide range of topics related to AI, with a focus on creating image classifiers.

Machine Learning Fundamentals Nanodegree
More advanced phase of AI Programming with Python. Focuses on Computer Vision Projects and Deployment using AWS SageMaker.

Machine Learning Specialization
Developed skills in building and training machine learning models, supervised and unsupervised learning, neural networks, decision trees, and ensemble methods.

Work Experience

03/2024 – present
Belgium

AI Solutions Engineer
Ariolas
- Developed and deployed AI-powered conversational digital humans.
- Designed staging infrastructure using a VPS that cost $72 annually achieving 95% annual cost reduction vs. AWS $1,392 VPS equivalent, delivering $1,320 annual savings for staging environment.

01/2024 – 03/2024
United States

Solutions Developer
Channel Automation LLC
- Built a GPT-powered self-scheduling chatbot that eliminated manual booking inefficiencies and helped close a $40,000 deal.
- Developed a FastAPI endpoint to integrate two platforms, hosted on a Hetzner VPS with Docker and secured via Cloudflare Tunnel.

09/2023 – 01/2024
Philippines

AI Integration Specialist
Capella BPO
- Developed the backend of an AI-powered healthcare chatbot using FastAPI, incorporating Retrieval-Augmented Generation (RAG) techniques to enhance the AI's response accuracy and relevance.
- Integrated the OpenAI GPT model into the healthcare chatbot, enhancing its ability to provide intelligent and context-aware responses.
- Designed and implemented a robust Azure CI/CD pipeline leveraging GitHub Actions, automating deployment and improving code integration for continuous delivery.

01/2023 – present
Philippines

Software | AI Engineer
Inniv8 Web Services
- Engineered an automated WordPress development and deployment process for 1,000+ brochure sites, reducing site creation time from an average of 10 hours of manual development to just 30 minutes through automation—saving over 9,500 hours using Python, Selenium, and APIs.
- Integrated AI (GPT models) to enhance content generation, improving efficiency 5x and enabling large-scale, hands-free website launches.
- Optimized development resources, allowing a 2-person team to handle a workload that would typically require 5+ developers, significantly reducing hiring costs while scaling operations efficiently.

Education

2017 – 2019
Angeles City, Philippines

ICT - Mobile App and Web Development
STI College Angeles

2019 – 2023
Angeles City, Philippines

Bachelor of Science in Information Technology
Holy Angel University

ROMEO C. VILLAMOR

MACHINE LEARNING ENGINEER

0975-068-4105

romeovillamor013220@gmail.com

San Isidro Magalang Pampanga

EDUCATION

ICT – Senior High School

Tinajero National High School

2013 - 2019

BS Information Technology

Pampanga State Agricultural University

2019-2023

SKILLS

- Web (HTML,CSS,JAVASCRIP,PHP)
- MY SQL Database Management
- Hardware Trouble Shooting
- Administrative & Financial Task
- System Support
- Android Development (JAVA)
- Basic Networking
- Machine Learning (Python, TensorFlow, scikit-learn)

LANGUAGE

English

About Me

Machine Learning Engineer with expertise in Python and data processing. Experienced in implementing ML solutions for business optimization while providing system support and database management, Currently applying ML techniques to improve operational efficiency across 100+ branches

WORK EXPERIENCE

Aug 2023 – Present

System Support Specialist

Balibago Waterworks System Inc.

- Provide hardware and system support remotely to 100+ branches, ensuring smooth operations.
- Manage and resolve support tickets efficiently.
- Train new employees on system features and processes to enhance productivity.
- Debug and troubleshoot system errors to maintain optimal functionality.
- Develop and maintain company websites as needed, utilizing PHP (Laravel), Tailwind CSS, JavaScript, and MySQL.
- Develop and maintain machine learning models for predictive maintenance and operational forecasting
- Perform basic network troubleshooting to resolve connectivity issues.
- Adjust and manage branch data in MySQL databases to ensure data accuracy and integrity.

SECONDARY ROLE/POSITONS

Accounting Custodian

- Manage and oversee IT department funds, ensuring proper allocation and utilization.
- Process reimbursements, cash advances, and budget requests in compliance with company policies.
- Handle the release and replenishment of department funds, maintaining accurate financial records.

Appendix C: Relevant Source Code

Dataset Generator Code: Extract_images

```
import os
import shutil
import numpy as np
import argparse
from cv2 import cv2
tileset_path =
os.path.abspath("tilesets/tileset.png")  #
Get full path
print("Using path:", tileset_path)

tileset = cv2.imread(tileset_path)

if tileset is None:
print(f"Error: Unable to read the
image at '{tileset_path}'")
else:
print("Image loaded
successfully!")

'''
This script is used to extract the
single sprites included in the tileset.png
in the same folder.
'''
def ExtractTiles(path):
tileset = cv2.imread(path)
if tileset is None:
print(f"Error: Unable to read the
image at '{path}'")
return  # Exit the function to
prevent further errors
tileset = tileset[..., ::-1]
y, x, z = tileset.shape  # y,x,z
lvl_wide = x//3
lvl_height = y//2
grid_size = (16, 16)  # y,x
for level in np.arange(4):
x_offset_lvl = 1 +
(lvl_wide+1)*(level % 3)
y_offset_lvl = 12 +
(level//3)*(37+136)
for y_i in
np.arange(136//grid_size[0]):
y_offset = (grid_size[0]+1)*y_i +
y_offset_lvl
for x_i in
np.arange((x//3)//grid_size[1]-1):
x_offset = (grid_size[1]+1)*x_i +
x_offset_lvl
if (y_i == 6 and x_i > 9):
continue
elif (y_i == 7 and x_i > 9):
sprite = tileset[y_offset-
grid_size[0]-1:y_offset +
grid_size[0]-1,
x_offset:x_offset+grid_size[0], :]
else:
sprite =
tileset[y_offset:y_offset+grid_size[0],
x_offset:x_offset+grid_size[0], :]
sprite_write =
cv2.cvtColor(sprite,
cv2.COLOR_RGB2BGR)
cv2.imwrite("Sprites/Sprite%d_
%d_%d.png" %
(level, x_i, y_i), sprite_write)
if __name__ == '__main__':
parser =
argparse.ArgumentParser(
"Extract tiles from tileset.")
parser.add_argument('--
path_tileset', '-t',
type=str,
default="tilesets/tileset.png",  #
Corrected
```

```
        required=False,
        help='Path to tileset file.',
        )
        FLAGS, _ =
parser.parse_known_args()
        dir = 'Sprites'
        if os.path.exists(dir):
        shutil.rmtree(dir)
        os.makedirs(dir)
        ExtractTiles(FLAGS.path_tileset
)
```

Dataset Generator Code: Generate_frames

```
        import os
        import shutil
        import multiprocessing
        import pandas as pd
        import numpy as np
        from cv2 import cv2
        import time
        import argparse
        from tqdm import tqdm
        from utils.generate_grid import
GridGenerator
        from utils.load_sprites import
SpriteLoader
        class FrameGenerator():
        ''' Super mario frames have a
dimension of
        256 x 240 x 3 (x,y,c)
        The idea is to place labels inside
a grid
        and then fill with the
corresponding image and
        its segmentation
        '''
        def __init__(self,
sprite_dataset="label_assignment/sprite_
labels_correspondence.csv", cores=1):
        '''Initializes the frame generator'''
        self.cores = cores
        self.sprites =
pd.read_csv(sprite_dataset, sep=',')
        self.labels =
self.sprites.Label.unique()
```

```
        self.background_colors = {0:
np.array(
        [147, 187, 236]), 1: np.array([0,
0, 0])}
        self.sprite_bg_color =
np.array([147, 187, 236])
        self.grid_h = 15
        self.grid_w = 17
        self.classes = {"default": 0,
        'floor': 1,
        'brick': 2,
        'box': 3,
        'enemy': 4,
        'mario': 5}
        self.classcolors = {"default": [0,
0, 0],
        'floor': [0, 0, 255],
        'brick': [127, 127, 0],
        'box': [0, 255, 0],
        'enemy': [255, 0, 0],
        'mario': [255, 255, 0]}
        def SetLevelSprites(self, level):
```

```
        '''This function loads sprites and
textures that will be used in the image.
        Level = 'xyz' with:
        x - Level tileset to use (not
relevant for grid generation)
        y - type of level:
        - 0 means default level, with
bushes and hills in the background
        - 1 means default level with trees
in the background
        - 2 means underground level. No
trees or bushes in the background
        - 3 means castle level (not
implemented)
        - 4 means mushroom level (not
implemented)
        - O means default level with
alternate bushes
        - I means default level with
alternate trees
        z - background color (not
relevant for grid generation)
        - 0 default blue
```

```
        - 1 black
        '''
        tileset = int(level[0])
        if level[1] == '2':
            tileset = 1
        self.floor =
SpriteLoader.loadFloor(tileset)
        self.box =
SpriteLoader.loadBox(tileset)
        self.brick =
SpriteLoader.loadBrick(tileset)
        self.pipe =
SpriteLoader.loadPipes(tileset)
        self.block =
SpriteLoader.loadBlock(tileset)

        if level[1] == '0' or level[1] ==
'1':
            tileset = 0
        elif level[1] == 'O' or level[1] ==
'I':
            tileset = 3
        else:
            tileset = 0
        self.hill =
SpriteLoader.loadHills(tileset)
        self.clouds =
SpriteLoader.loadClouds(0)
        self.bushes =
SpriteLoader.loadBushes(tileset)
        self.trees =
SpriteLoader.loadTrees(tileset)
        self.spipe =
SpriteLoader.GenerateSSGT(
        self.pipe, self.classcolors['floor'])
        self.seg_floor =
SpriteLoader.SpriteSSGT(
        self.floor,
self.classcolors['floor'])
        self.sbox =
SpriteLoader.SpriteSSGT(self.box,
self.classcolors['box'])
        self.sbrick =
SpriteLoader.SpriteSSGT(
        self.brick,
self.classcolors['brick'])
```

```
        self.sblock =
SpriteLoader.SpriteSSGT(
        self.brick,
self.classcolors['floor'])
        def LoadSprites(self, level):
        '''Load sprites for mario, enemies
and generates their ground truth.'''
        if level[1] == '2':
        tileset = 1
        else:
        tileset = 0
        self.mushroom =
SpriteLoader.loadGoombas(tileset)
        self.smushroom =
SpriteLoader.GenerateSSGT(
        self.mushroom,
self.classcolors['enemy'])
        self.koopa =
SpriteLoader.loadKoopa(tileset)
        self.skoopa =
SpriteLoader.GenerateSSGT(
        self.koopa,
self.classcolors['enemy'])
        self.piranha =
SpriteLoader.loadPiranha(tileset)
        self.spiranha =
SpriteLoader.GenerateSSGT(
        self.piranha,
self.classcolors['enemy'])
        self.mario =
SpriteLoader.loadMario()
        self.smario =
SpriteLoader.GenerateSSGT(
        self.mario,
self.classcolors['mario'])
        def generate_frame(self,
level='000', grid=[]):
        if grid == []:
        grid = self.generate_grid(level)
        self.SetLevelSprites(level)
        self.LoadSprites(level)
        if level[1] == '2':
        bg_color = 1
        else:
        bg_color = int(level[2])
```

```
        self.background_color =
self.background_colors[bg_color]
        frame = np.zeros((16*15,
16*self.grid_w, 3))
        frame = frame +
self.background_color
        sframe = np.zeros((16*15,
16*self.grid_w, 3))
        classframe = np.zeros((16*15,
16*self.grid_w))
        missing_right = False  # for
piranha
        for row in
np.arange(grid.shape[0]):
        frow = row * 16  # index iterator
        for column in
np.arange(grid.shape[1]):
        fcol = column*16  # index
iterator
        if grid[row, column, 0] ==
'[background]':
        for i in np.arange(16):
        for j in np.arange(16):
        frame[frow+i, fcol+j, :] =
self.background_color
        sframe[frow+i, fcol+j] = [0, 0, 0]
        elif grid[row, column, 0][1:6] ==
'cloud':
        for i in np.arange(16):
        for j in np.arange(16):
        if (self.clouds[grid[row, column,
0]][i, j] != self.sprite_bg_color).any():
        frame[frow+i, fcol+j,
        :] = self.clouds[grid[row,
column, 0]][i, j]
        elif grid[row, column, 0][1:5] ==
'hill':
        for i in np.arange(16):
        for j in np.arange(16):
        if (self.hill[grid[row, column,
0]][i, j] != self.sprite_bg_color).any():
        frame[frow+i, fcol+j,
        :] = self.hill[grid[row, column,
0]][i, j]
        else:
        for i in np.arange(16):
```

```
            for j in np.arange(16):
            frame[frow+i, fcol+j] = [255,
255, 255]
            sframe[frow+i, fcol+j] = [255,
255, 255]
            if grid[row, column, 1] ==
'[floor]':
            for i in np.arange(16):
            for j in np.arange(16):
            frame[frow+i, fcol+j, :] =
self.floor[i, j]
            sframe[frow+i, fcol+j, :] =
self.seg_floor[i, j]
            classframe[frow+i, fcol+j] =
self.classes['floor']
            elif grid[row, column, 1] ==
'[box]':
            for i in np.arange(16):
            for j in np.arange(16):
            if (self.box[i, j, :] !=
self.sprite_bg_color).any():
            frame[frow+i, fcol+j, :] =
self.box[i, j]
            sframe[frow+i, fcol+j, :] =
self.sbox[i, j]
            classframe[frow+i, fcol +
            j] = self.classes['box']
            elif grid[row, column, 1] ==
'[brick]':
            for i in np.arange(16):
            for j in np.arange(16):
            if (self.brick[i, j, :] !=
self.sprite_bg_color).any():
            frame[frow+i, fcol+j, :] =
self.brick[i, j]
            sframe[frow+i, fcol+j, :] =
self.sbrick[i, j]
            classframe[frow+i, fcol +
            j] = self.classes['brick']
            elif grid[row, column, 1][1:5] ==
'bush':
            for i in np.arange(16):
            for j in np.arange(16):
            if (self.bushes[grid[row, column,
1]][i, j, :] != self.sprite_bg_color).any():
            frame[frow+i, fcol+j,
```

```
            :] = self.bushes[grid[row,
column, 1]][i, j]
            elif grid[row, column, 1][1:5] ==
'tree':
            for i in np.arange(16):
            for j in np.arange(16):
            if (self.trees[grid[row, column,
1]][i, j, :] != self.sprite_bg_color).any():
            frame[frow+i, fcol+j,
            :] = self.trees[grid[row, column,
1]][i, j]
            if grid[row, column, 2][1:5] ==
'mush':
            for i in np.arange(16):
            for j in np.arange(16):
            if (self.mushroom[grid[row,
column, 2]][i, j] !=
self.sprite_bg_color).any():
            frame[frow+i, fcol+j,
            :] = self.mushroom[grid[row,
column, 2]][i, j]
            sframe[frow+i, fcol+j,
            :] = self.smushroom[grid[row,
column, 2]][i, j]
            classframe[frow+i, fcol +
            j] = self.classes['enemy']
            if grid[row, column, 2][1:6] ==
'koopa':
            for i in np.arange(32):
            for j in np.arange(16):
            row_off = frow+i-16
            if (self.koopa[grid[row, column,
2]][i, j] != self.sprite_bg_color).any():
            frame[row_off, fcol+j,
            :] = self.koopa[grid[row, column,
2]][i, j]
            sframe[row_off, fcol+j,
            :] = self.skoopa[grid[row,
column, 2]][i, j]
            classframe[row_off, fcol +
            j] = self.classes['enemy']
            if grid[row, column, 2][1:8] ==
'piranha':
            if missing_right == False:
            y = np.random.randint(1, 21)
```

```
            piranha_height =
np.random.choice([0, y])
            if grid[row+1, column, 2] ==
'[pipe_tl]':  # then only print half piranha
            for i in np.arange(32):
            for j in np.arange(8):
            row_off = frow+i-16 +
piranha_height
            col_off = fcol+j + 8
            if (self.piranha[grid[row, column,
2]][i, j] != self.sprite_bg_color).any():
            frame[row_off, col_off,
            :] = self.piranha[grid[row,
column, 2]][i, j]
            sframe[row_off, col_off,
            :] = self.spiranha[grid[row,
column, 2]][i, j]
            classframe[row_off,
            col_off] = self.classes['enemy']
            missing_right = True
            if grid[row+1, column, 2] ==
'[pipe_tr]':  # then only print half piranha
            for i in np.arange(32):
            for j in np.arange(8):
            row_off = frow+i-16 +
piranha_height
            col_off = fcol+j
            if (self.piranha[grid[row, column,
2]][i, j+8] != self.sprite_bg_color).any():
            frame[row_off, col_off,
            :] = self.piranha[grid[row,
column, 2]][i, j+8]
            sframe[row_off, col_off,
            :] = self.spiranha[grid[row,
column, 2]][i, j+8]
            classframe[row_off,
            col_off] = self.classes['enemy']
            missing_right = False
            if grid[row, column, 2][1:5] ==
'pipe':
            for i in np.arange(16):
            for j in np.arange(16):
            if (self.pipe[grid[row, column,
2]][i, j] != self.sprite_bg_color).any():
            frame[frow+i, fcol+j,
```

```
            :] = self.pipe[grid[row, column,
2]][i, j]
            sframe[frow+i, fcol+j,
            :] = self.spipe[grid[row, column,
2]][i, j]
            classframe[frow+i, fcol +
            j] = self.classes['floor']
            if grid[row, column, 2][1:6] ==
'block':
            for i in np.arange(16):
            for j in np.arange(16):
            if (self.block[i, j] !=
self.sprite_bg_color).any():
            frame[frow+i, fcol+j, :] =
self.block[i, j]
            sframe[frow+i, fcol+j, :] =
self.sblock[i, j]
            classframe[frow+i, fcol +
            j] = self.classes['floor']
            if grid[row, column, 2][1:6] ==
'mario':
            for i in np.arange(16):
            for j in np.arange(16):
            if (self.mario[grid[row, column,
2]][i, j] != self.sprite_bg_color).any():
            frame[frow+i, fcol+j,
            :] = self.mario[grid[row, column,
2]][i, j]
            sframe[frow+i, fcol+j,
            :] = self.smario[grid[row,
column, 2]][i, j]
            classframe[frow+i, fcol +
            j] = self.classes['mario']
            return frame, sframe, classframe
            def generate_grid(self, level):
            grid_gen = GridGenerator()
            return
grid_gen.GenerateGrid(level)
            def GenerateSamples(self,
init_filenumber, end_filenumber, seed,
w_tqdm=False):
            np.random.seed(seed)
            files = None
            if w_tqdm == True:
```

```
        files =
tqdm(np.arange(init_filenumber,
end_filenumber))
        else:
        files =
np.arange(init_filenumber,
end_filenumber)
        for i in files:
        x = '0'  #
np.random.choice(['0','1'])
        y = np.random.choice(['0', '1', '2',
'O', 'I'])
        z = np.random.choice(['0', '1'],
p=[.8, .2])
        level = x + y + z
        frame, sframe, classframe =
self.generate_frame(level)
        frame =
cv2.cvtColor(frame.astype(np.uint8),
cv2.COLOR_RGB2BGR)
        cv2.imwrite("dataset/PNG/%d.pn
g" % (i), frame)
        sframe =
cv2.cvtColor(sframe.astype(np.uint8),
cv2.COLOR_RGB2BGR)
        cv2.imwrite("dataset/Segmentati
on/%d.png" % (i), sframe)
        cv2.imwrite("dataset/Labels/%s.
png" % (i), classframe)
        def GenerateDataset(self,
samples):
        '''Generates a dataset of a given
size.'''
        start = time.time()
        threads = self.cores
        seeds = np.random.randint(100,
size=threads)
        dir = 'dataset'
        if os.path.exists(dir):
        shutil.rmtree(dir)
        os.makedirs(dir)
        os.makedirs('dataset/PNG/')
        os.makedirs('dataset/Segmentatio
n/')
        os.makedirs('dataset/Labels/')
```

```
        if self.cores == 1:
        level = np.random.choice([0, 1])
        self.GenerateSamples(0, samples,
level, w_tqdm=True)
        else:
        step = samples//self.cores
        ppool =
multiprocessing.Pool(threads)
        ranges =
step*np.arange(self.cores+1)
        ppool.starmap(self.GenerateSam
ples, zip(
        ranges[:-1], ranges[1:], seeds))


        end = time.time()
        print("Elapsed time:", end-start)
        if __name__ == '__main__':
        parser =
argparse.ArgumentParser(
        "Generate a semantic
segmentation dataset with synthetic
super mario frames")
        parser.add_argument('--cores', '-
c',
        type=int,
        default=1,
        required=False,
        help='How many cores to use,
speeds up generation. Set to 1 if using
windows',
        )

        parser.add_argument('--samples',
'-s',
        type=int,
        default=0,
        required=True,
        help='How many images to
generate.',
        )
        FLAGS, _ =
parser.parse_known_args()
        framegen =
FrameGenerator(cores=FLAGS.cores)
```

```
        framegen.GenerateDataset(FLA
GS.samples)
```